

强化学习入门（小鸭子版）：从策略梯度到 PPO，让一只 800 克的小鸭子自己学会动

目录

1 从「教它走路」到「给它打分」	4
1.1 先看它在做什么	4
1.2 为什么这一讲不从示教走	4
1.3 换一种监督信号：不给答案，只给分数	6
1.4 输入输出到底是什么：61 个数进，14 个数出	7
1.5 一步的分数怎么给：奖励表与回报	9
1.6 策略与价值：贯穿全篇的两个量	11
1.7 训练数据是它自己跑出来的	12
1.8 这一讲要走的路线	12
1.9 术语对照表	13
2 强化学习方法分类	14
2.1 第一个划分：学不学环境模型	15
2.2 第二个划分：学什么——基于价值还是基于策略	15
2.3 Actor-Critic：横跨两类的混合架构	15
2.4 第三个划分：采数据的策略和学的策略是不是同一个	16
2.5 把常见算法放上这张地图	16
2.6 本节小结	17
3 认识这只小鸭子：从硬件到仿真	17
3.1 它是什么：800 克、25 厘米、两条腿	18
3.2 骨架与关节：14 个舵机怎么分布，第 15 个在哪	19
3.3 大脑与神经：板子、总线、传感器、电池	20
3.4 它平时怎么被驱动：50 赫兹的控制环	20
3.5 仿真里的这只鸭子：三个模型，各有各的用处	21
3.6 舵机不是理想的 PD：BAM 执行器模型	21
3.7 每个回合都不完全一样：域随机化	21

3.8 mlab：一次开几千只鸭子的练功房	22
3.9 代码地图：一节讲义对一个模块	23
4 v1 REINFORCE：先自己试，试好了就多做	24
4.1 直觉：没有标准答案，怎么知道该往哪调	24
4.2 好动作加概率，坏动作减概率	24
4.3 要写成式子的话	25
4.4 一轮训练长什么样：采一批、算回报、更新一次	26
4.5 代码走读：v1 版训练循环	27
4.6 实验：它真能走起来，但走得难看	29
4.7 毛病在哪：回报的噪声淹没了学习信号	30
4.8 本节小结	30
5 v2 A2C：请一位评分员	32
5.1 直觉：跟「平均水平」比，不是跟零比	32
5.2 评分员是什么：价值与优势	32
5.3 要写成式子的话	33
5.4 代码走读：v1 到 v2 改了什么	34
5.5 实验：动作幅度打开了，但时不时被推坏	36
5.6 毛病在哪：更新的步子没人管	37
5.7 本节小结	38
6 v3 PPO：给更新的步子加护栏	39
6.1 直觉：一次别改太多	39
6.2 护栏怎么加：概率比值与 clip	39
6.3 顺手省数据：一段数据反复训几轮	40
6.4 学习率自己调：动得太猛就减速	41
6.5 要写成式子的话	41
6.6 代码走读：v2 到 v3 改了什么	42
6.7 实验：三个算法同台，行走任务收口	43
6.8 本节小结	45
7 一套 PPO，切换任务配置学不同动作	45
7.1 谱系总览：一个仓里三十三个任务，算法一行没改	46
7.2 脚下装上轮子：轮滑	46
7.3 原地转起来：旋转	47
7.4 低头，用嘴尖碰到地上的东西：取物	49
7.5 翻过去再站起来：前滚翻	49
7.6 目标可以完全不在观测里	51
7.7 “慢才是最优”怎么写进奖励	52
7.8 写奖励的几条规矩	52

7.9 一个身体，多个策略	52
7.10 奖励表和课程表都在环境这一侧	53
8 本讲小结	54
8.1 一条演进链：每一步为什么发生	54
8.2 一张小结表：同一套算法，几个不同的动作	54
8.3 还能往哪走	55
附录：复现这一讲	55
A.1 代码、权重与产物	55
A.2 环境与硬件	55
A.3 训练与评测口径	56
A.4 参考	56



图 1: 四只小鸭子摆在不同姿态下。**舵机和连杆是外露的**，从这张图上就能一节节数过去：髋、膝、踝各一组，脖子与头另有四个。这也是它便宜的原因之一——没有外壳要做。（厂商照片，Pollen Robotics）

一只 800 克、25 厘米高的双足小鸭子，站在一块什么都没有的平地上。它有 14 个舵机，每秒被指挥 50 次。没有人教它怎么走。

它学会的这些动作全都是自己试出来的：按指令朝任意方向走，脚下换成轮子后滑行，原地飞快旋转，低头用嘴尖碰到地面上的东西，向前翻一个跟头再站回来。**这里没有一个动作是人教的**。没有示教数据，没有动作捕捉，没有任何一条“这一步该做什么”的标准答案，只有一个能打分的函数，和大量的失败。

这一讲讲的就是这件事怎么可能。我们会先把“学走路”翻译成强化学习的语言，建立一张不会混淆的方法分类地图，然后沿着地图上一条主线，从最朴素的做法出发，每讲完一个算法就在同一个任务上跑给你看——看到它的毛病，再引出下一个算法——一路走到工业界在用的那套配方。最后把同一套配方原样换到别的动作上，看它能学出多少花样。

全篇只在仿真里进行。

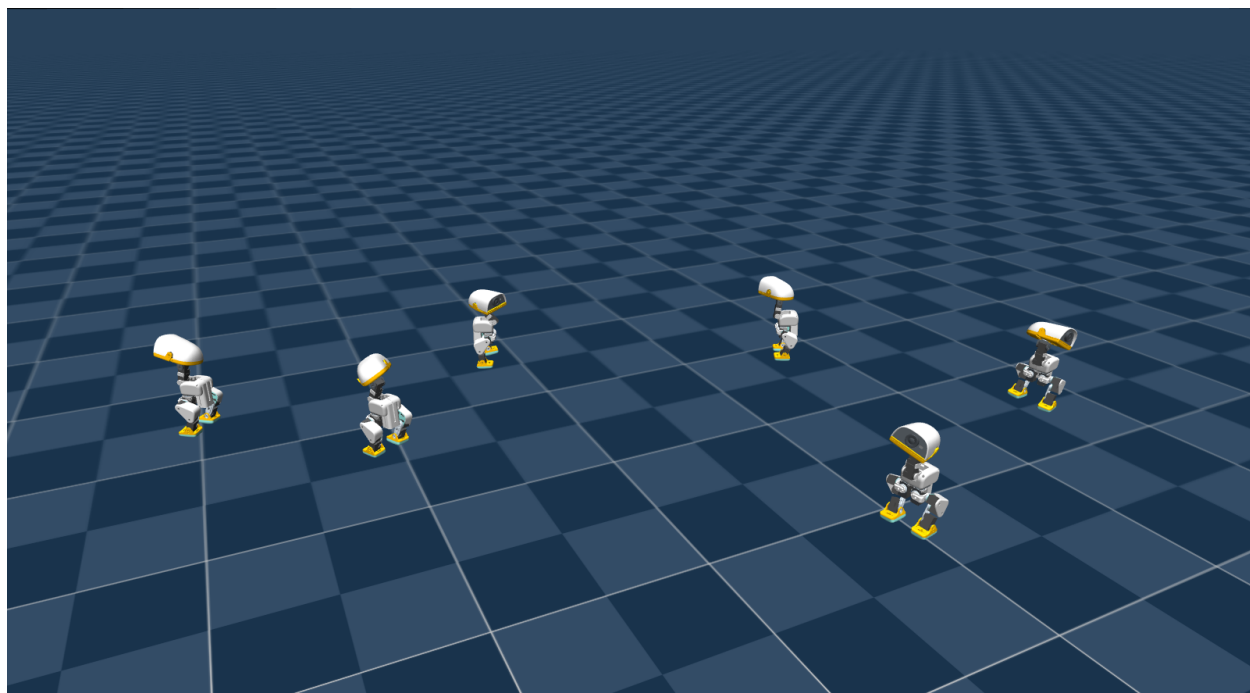


图 2: 训练中的一屏。这张图是叠画出来的，不是一个场地里的实况。并行训练是「多个环境、每只鸭子一个环境」，每个环境是一份独立的物理世界，鸭子之间既看不见也碰不到；渲染器把邻近几个环境的鸭子追加进同一张画面，才有了这个效果。真实训练时同时跑的是 4096 个。

1 从「教它走路」到「给它打分」

1.1 先看它在做什么

一只 800 克、25 厘米高的小鸭子站在一块空平地上。

它身上有 14 个舵机（两条腿各 5 个，脖子和头 4 个）。每 20 毫秒，一个神经网络读一遍它此刻的姿态，给这 14 个舵机各下一个目标角度；舵机去追那个角度，物理世界演进 20 毫秒，网络再读一遍。一秒钟五十轮，中间没有人。

现在给它一个任务：**按指令的速度在平地上走**。往前多快、往侧多快、转多快，每个回合随机给一组；走稳了、跟上了、没摔，就给分。摔了就重置，从站姿再来一遍。

这一讲要讲的就是这个网络怎么从”完全不会”变成”稳稳地走”。下面先把上面这段白话里的每个东西，换成强化学习的说法。

1.2 为什么这一讲不从示教走

教机器人做事最直接的办法是**示范给它看**：人操作一遍，把每一时刻的动作记下来，让网络去模仿（这就是**行为克隆**，behavior cloning）。这套办法在有些地方管用——动作慢、自由度少、人能直接带着

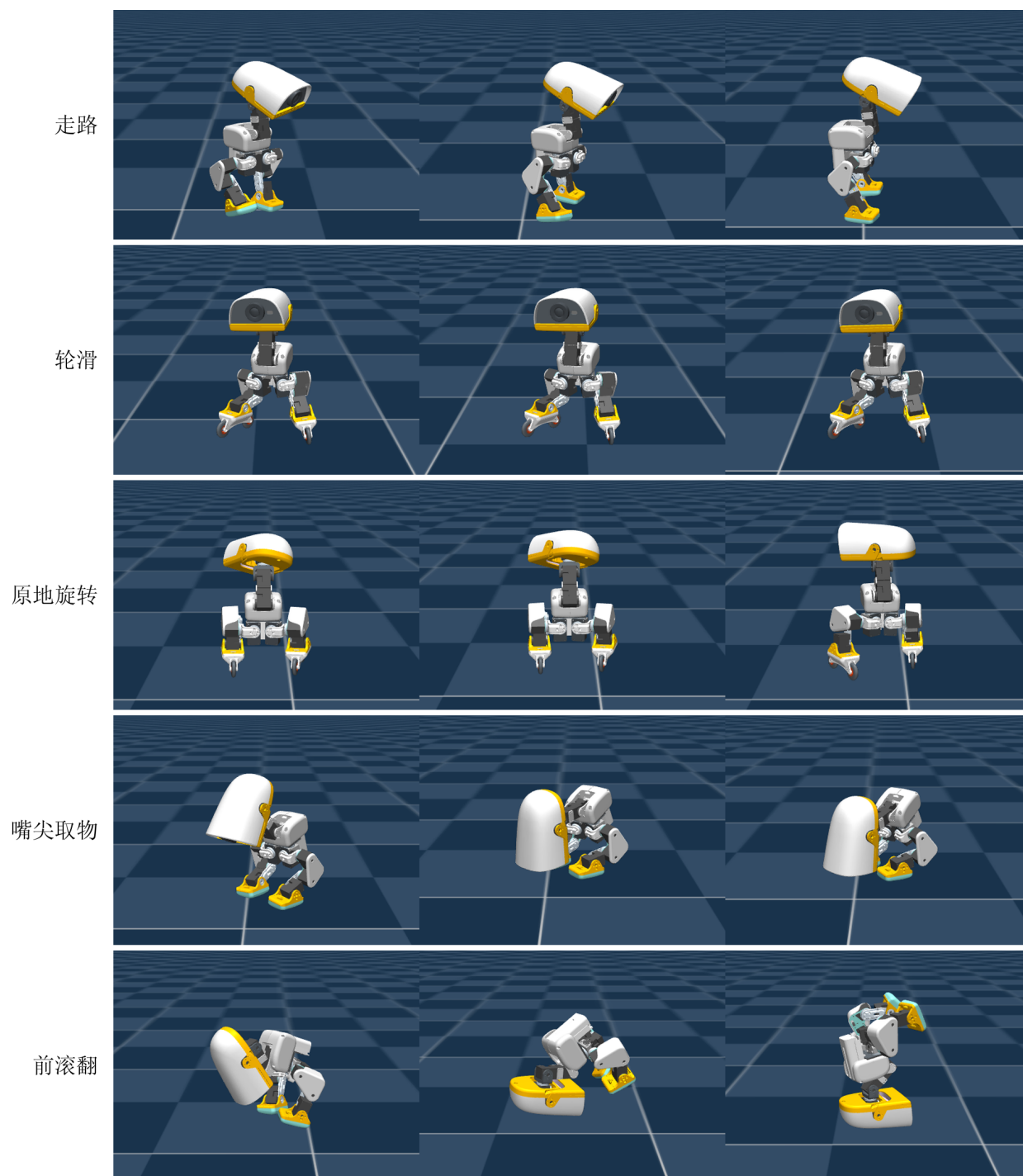


图 3: 五个动作, 一行一个; 行内三张是同一个动作的三个瞬间, 按时间顺序。五个动作复用同一个 PPO 训练器、同一份 61 进 14 出的接口; 动的全在环境那一侧——奖励表、指令、课程表、重置与终止条件, 轮滑那档还换了本体(脚 → 被动轮)。

做的，一步一步录下来就行。

放到这只鸭子身上，“少量离线示范 + 行为克隆”这个最朴素的版本有三处要克服的地方。

一、示范本身不好采。走路不是一个姿势，是 14 个关节以每秒 50 次的频率协调发力。人有两条腿，但人的腿和这只鸭子的腿在长度比例、质量分布、关节限位上都不一样，“我怎么走”照搬过去它站不住。就算你想逐关节遥操作，一只手也管不了 14 个自由度，更别说要在毫秒级上保持配合。

二、示范不告诉你哪种更好。假设真采到了一段能走的示范，行为克隆的目标是“贴近这段示范”。但示范里没有“这样走比那样走更省力”、“这个步幅更抗推”这类信息——监督信号只回答“这一步该做什么”，不回答“这一步做得好不好”。想要机器人自己找到比示范者更好的做法，纯行为克隆里没有这个概念。

三、走错了没人救（分布偏移）。闭环控制里误差会滚雪球：机器人的动作影响下一时刻的状态，一旦偏离到示范数据没覆盖过的姿态，网络就两眼一抹黑——那里没有标签。而一只 25 厘米高、重心偏上的双足机器人，偏离到“没见过的姿态”只需要一次跟跑。

三处的根源是同一个：示教给的是“该做什么”，而我们需要的是“做得怎么样”。

这三处都有对策，都在模仿学习这条线上：DAgger 让专家在策略跑出来的状态上补标注，治的是第三处；动作重定向（retargeting）把人的动作映射到这具身体的关节上，治的是第一处；逆强化学习与对抗式模仿从示范里反推出一个奖励，再用它优化，治的是第二处；还有更常见的一种做法是先用示范预训练、再用强化学习微调，两条路一起走。

所以这一讲从零开始做强化学习，不是因为示范这条路走不通，而是因为它要讲的恰恰是“做得怎么样”这件事怎么变成一个可优化的目标。上面三处正好是这个目标的三个用处，把它们摆出来，是为了让后面每一个奖励项的存在理由都有个落点。

1.3 换一种监督信号：不给答案，只给分数

“做得怎么样”用一个标量就能表达：奖励（reward，记作 r ）。

强化学习把世界切成两半：智能体（agent）和环境（environment）。智能体就是那个神经网络；除它之外的一切——机器人的身体、地面、重力、指令是怎么生成的——都算环境。

一次交互长这样：

1. 环境处在某个状态（state, x_t ），把其中策略看得到的那部分——观测 o_t ——交给智能体（1.4 节说清这 61 个数具体是什么）；
2. 智能体选一个动作（action, a_t ）；
3. 环境执行这个动作，物理演进一个控制周期（20 毫秒），到达新状态 x_{t+1} ，并给出这一步的奖励 r_t ；
4. 回到第 1 步。



图 4: 智能体与环境的闭环。环境把观测 o_t 交给智能体，智能体出动作 a_t ，环境执行完给出新的观测和这一步的奖励 r_t 。与 1.2 节那个离线行为克隆相比，差别不只是多出来的那条奖励线：**强化学习的数据由策略自己与环境交互产生、优化目标是期望回报**；行为克隆的数据由专家给定、目标是复现示范动作。多出来的这条线是入口，训练目标与数据来源的不同才是分野。

这样一条 $(s_0, a_0, r_0, s_1, a_1, r_1, \dots)$ 的序列叫一条**轨迹** (trajectory)。轨迹不会无限长，它因为两种原因结束：

- **终止** (termination)：出事了 —— 鸭子摔倒、姿态坏到没救。
- **截断** (truncation)：时间到了 —— 回合有最大长度，到点就重置，不代表失败。

一条从开始到结束的轨迹叫一个**回合** (episode)。走路这个任务的回合是 1000 步，也就是 20 秒。区分终止和截断很要紧，第 6 节讲 PPO 时会用到：摔倒了就是摔倒了，后面没有价值可言；而时间到了被打断，机器人本来还能继续挣分 —— 这两种情况在算“未来还能拿多少分”时不能一样对待。

换成打分之后，两件事跟着变了。

一是监督信号的性质。模仿学习的每条样本都带着动作标签，属于监督学习；强化学习没有标签，只有事后的评分，而且评分往往是延迟的 —— 现在这一步好不好，可能要几十步之后摔了才知道。

二是数据的来源。模仿学习的数据集是事先采好的、静态的；强化学习的训练数据是机器人自己跑出来的。这一点在 1.7 节展开。

1.4 输入输出到底是什么：61 个数进，14 个数出

那个网络叫**策略** (policy，记作 π_θ ， θ 是它的参数)。它做的事只有一件：**读一串数，写一串数**。

- 读进去的叫**观测** (observation, o_t), 61 个浮点数;
- 写出来的叫**动作** (action, a_t), 14 个浮点数。

这两串数具体是什么量、什么单位，值得说清楚 —— 它决定了这个策略在真机上要接什么。

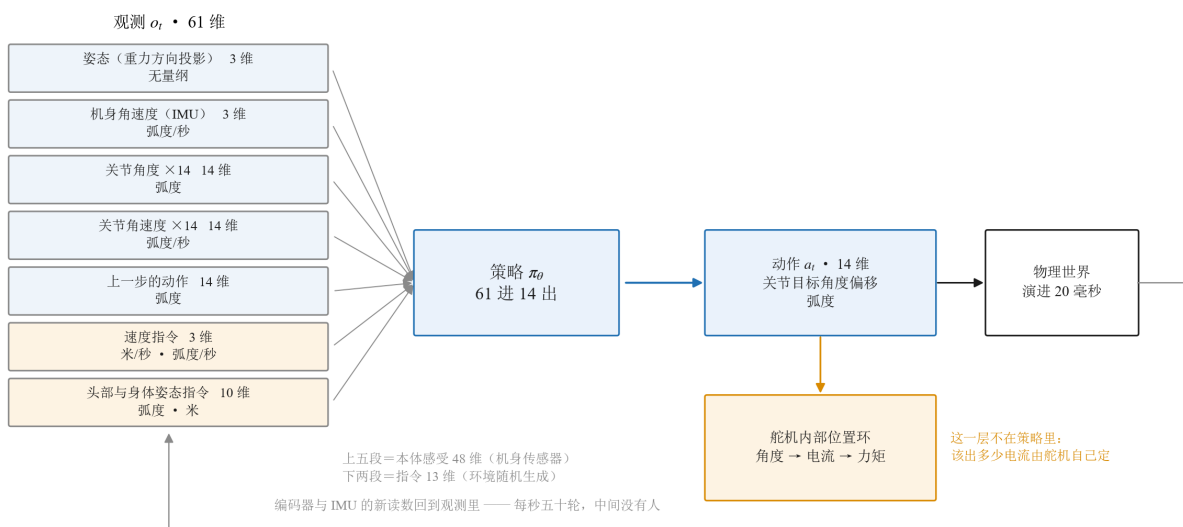


图 5: 策略两侧的接口。左边 61 维观测按来源分成四段，右边 14 维动作是关节目标角度；策略不给电压也不给力矩，从角度到电流那一步在舵机内部的位置环里完成。图里的单位就是代码里实际用的单位。

输入这 61 个数分成两部分。前 48 个是**本体感受** (proprioception)，机器人对自己身体的感觉，全部来自机身上的传感器：

段	维数	是什么量	单位
姿态（重力方向在机身坐标系里的投影）	3	无量纲的方向余弦	—
机身角速度（IMU 陀螺仪）	3	角速度	弧度/秒
关节角度（14 个舵机的编码器读数）	14	角度相对站立姿态的偏移	弧度
关节角速度	14	角速度	弧度/秒
上一步下发的动作	14	目标角度偏移	弧度

后 13 个是**指令** (command) —— 这一回合要它做什么，由环境随机生成：

指令	维数	是什么量	单位
线速度与转向	3	前进、侧移、转向	米/秒、米/秒、弧度/秒
头部姿态	4	颈与头四个关节的目标偏移	弧度
身体姿态	6	躯干位置与朝向偏移（走路任务里权重极小）	米、弧度

输出这 14 个数全是同一种量：**关节目标角度的偏移**（弧度），加到站立姿态上就是要去的角度。策略

不直接给力矩，也不直接给电压。它说“左膝去到 +0.3 弧度”，舵机内部的位置环去追这个目标，该出多少电流由舵机自己决定。这一层间接让学习容易得多——策略不必学习力学，只需要学“摆什么姿势”。

14 个关节这么分布：

编号	部位	关节
0-4	左腿	髋偏航、髋滚转、髋俯仰、膝、踝
5-8	颈与头	颈俯仰、头俯仰、头偏航、头滚转
9-13	右腿	髋偏航、髋滚转、髋俯仰、膝、踝

每个舵机能出的力矩大约 1 牛·米。这个数字后面很关键：它小得需要认真对待，第 3 节会解释为什么不能把它当成一个理想的位置伺服。

两件事现在先记下，第 7 节会回收：

- **所有任务共用同一份 61 维观测布局。**某个任务用不到某个指令槽位时，那几个数字被填零而不是删掉。这看起来浪费，实则有意——它让同一个身体上可以随时切换不同的策略，因为输入格式一样。
- **状态、观测、特权观测**是三个不同的量，本篇分开写。环境的完整状态记 x_t ——它包含地面摩擦系数、电池当前电压这些策略看不到的东西。actor 拿到的那 61 个数是**观测** o_t ，只是 x_t 的一部分（所以这个任务是**部分可观测的**， o_t 不是完整的 Markov 状态）。训练时还有第三个：critic 能看到仿真器里的额外信息，那份记**特权观测** c_t （第 5 节展开）。于是策略写作 $\pi_\theta(a_t | o_t)$ 、价值函数写作 $V_\phi(c_t)$ 。

1.5 一步的分数怎么给：奖励表与回报

奖励 r_t 是**每一步都有的**即时分数，不是等回合结束才给。但要最大化的不是某一步，而是**长期累计**——很多好动作的收益是延迟的：为了迈出稳的一步，此刻可能要先把重心压低，那一瞬间的分数并不高。

于是引入**回报** (return, R_t)：从第 t 步往后，所有奖励的折扣累加

$$R_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots = \sum_{k \geq 0} \gamma^k r_{t+k}$$

其中 $\gamma \in (0, 1)$ 叫**折扣因子** (discount factor)，本篇取 0.99。它给“多远的未来还算数”定一个尺度：越接近 1 越看重远期，越小越短视。取 0.99 意味着大约 100 步（2 秒）之后的奖励还剩三分之一的分量——对走路这种任务，两秒足够涵盖好几个步态周期。折扣还有个技术上的好处：它让无限长的和收敛。

到这里定义都齐了，但“奖励”听起来还是抽象。实际上它**不是一个函数，而是一张加权项表**：环境每一步把表里每一项各算一遍，乘上权重，加起来，就是这一步的 r_t 。

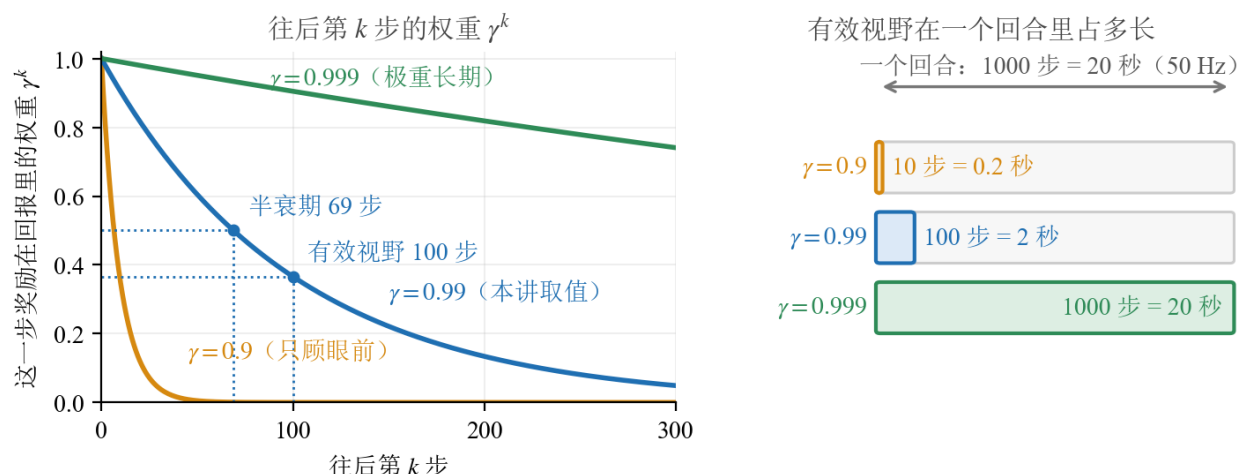


图 6: 折扣因子 γ 决定它看多远。左：往后第 k 步的奖励在回报里的权重 γ^k ；右：换算成“有效视野在一个回合里占多长”。本篇取 $\gamma = 0.99$ —— 半衰期 69 步、有效视野约 100 步，而走路一个回合是 1000 步（20 秒 / 50 赫兹）。取 0.9 就只顾眼前 0.2 秒，取 0.999 则把整个回合都算进来。

走路这个任务的表一共 **16 项**（下表按权重绝对值排，从环境配置里逐项读出来的，本篇只读它、不改它）：

要它做的	权重	不许做的	权重
腾空时间落在 0.125–0.300 秒	+3.0	抬脚高度偏离目标	−2.0
跟上线速度指令	+2.0	关节顶到行程限位	−1.0
跟上转向指令	+2.0	腿撞到躯干	−1.0
躯干保持直立	+2.0	摆动腿抬得过高	−0.25
跟上头部姿态指令	+2.0	动作变化太快	−0.1（见下面第三条）
腿关节贴近站立姿态	+1.0	脚打滑	−0.1
		躯干角速度过大	−0.05
		整机角动量过大	−0.02

先看这张表的形状：要它做的 6 项、不许做的 8 项，而“不许做的”里权重最大的一项（抬脚高度偏离目标，−2.0）与最小的一项（整机角动量，−0.02）差了两个数量级。

这个量级差不是随手给的：越是“物理上不得不做”的事（动态动作必然带来角动量），权重就要给得越小，否则它会把动作本身掐死。

不过权重之比不等于贡献之比 —— 每一项实际扣掉多少分，还取决于那一项原始量的数值范围与量纲（角动量的数值本来就比“抬脚高度误差”大得多）、有没有归一化、有没有门限。看权重能看出**设计者的意图排序**，看不出各项在总分里的实际占比；后者要把每一项的实测取值打出来才知道。

三个机制值得单独讲清，因为它们解释了这张表为什么这么写。

一、跟踪型的项按误差算分，形状是一条钟形曲线。

指令说“往前 0.4 米/秒”，它实际走了 0.35 —— 差 0.05，该扣多少分？最直接的写法是照误差成比例扣：差 0.05 扣一点，差 1.0 就扣二十倍。问题出在极端情况：鸭子被绊一下、瞬间速度差了 2 米/秒，那一步能扣掉几十分，把前面稳稳走的几百步全抹平。于是它学到的第一件事会是“千万别出现大误差”，而不是“走稳”。

这个任务的写法是把误差 e 送进 $\exp(-e^2/\sigma^2)$ ：

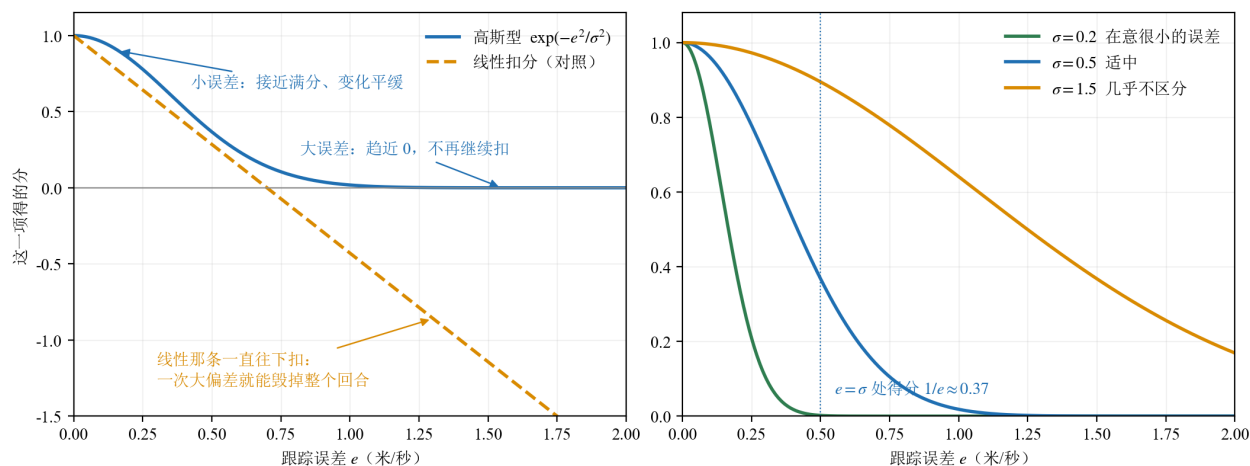


图 7: 跟踪型奖励为什么是钟形。左：与照误差成比例扣分相比，钟形在小误差处平缓、在大误差处贴住 0 —— 一次大偏差最多让这一项归零，扣不出一个大坑。右： σ 是“多大的误差你还在意”这个旋钮，取大了 0.2 与 0.8 的误差得分差不多，策略就懒得把最后那点误差磨掉。

左图那两条线的差别就是一句话：**钟形的最坏情况是这一项得 0 分，成比例那条没有下界**。右图那个 σ 定的是“多大的误差你还在意”，不是“最大可能的误差”。

二、**有些项只在特定条件下才算**。拿“脚打滑”举例：它只在速度指令的模长超过 0.01 时才计算。理由在指令本身 —— 指令是“站住不动”的时候，脚**就该**死死踩在地上，这时候按打滑扣分，等于因为它做对了而扣它分。“抬脚高度”那项挂的是同一个门限，理由也一样：站住不动的时候本来就不该抬脚。这类项在表里不是一个恒定的惩罚，而是“先看这一步该不该管，再决定算不算”。

三、**有一项的权重随训练推进变**。“动作变化太快”起初只有 -0.1 ，分段往上加，到第 1500 次迭代才到 -1.0 。为什么不一开始就给 -1.0 ？因为那样有一个分数更高的解：**一动不动**。站着不动的鸭子动作变化率是 0、一分不扣；而试着迈步的鸭子每一步都在被扣，它还没学会怎么把这些分从“走得稳”里挣回来。先让它敢动，等步子出来了，再要求它迈得优雅。

这三条不是这个任务特有的技巧，而是写奖励的通用规矩。第 7 节会在多个任务的奖励表并排之后，把这类规矩集中讲一遍。

1.6 策略与价值：贯穿全篇的两个量

策略 $\pi_\theta(a | o)$ ：在观测 o 下选择动作 a 的概率。注意它是一个**概率分布**而不是一个确定的动作 —— 这一点很重要，因为强化学习必须**探索** (exploration)：如果策略每次都输出同一个动作，它永远不会知

道别的动作是不是更好。本篇的策略输出一个高斯分布的均值，标准差是一组可训练的参数；训练时从这个分布里采样（带探索），评测时直接取均值（不带探索，成绩才可复现）。

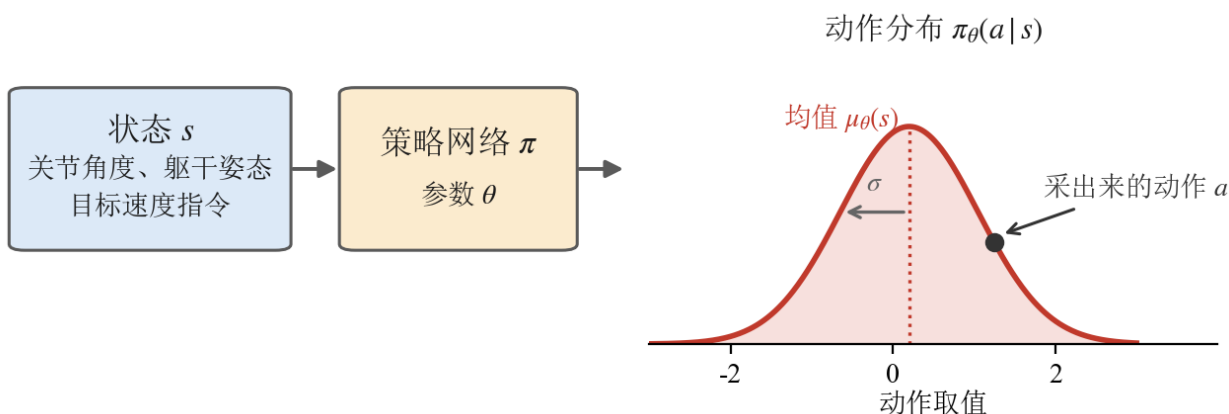


图 8: 策略在连续动作上输出的不是一个数，是一个分布：网络给出均值 $\mu_{\theta}(s)$ 与标准差 σ ，真正执行的动作是从这个高斯里采出来的。 σ 就是探索的幅度——第 4 节那个熵奖励调的正是它。

价值函数 (value function) $V(c)$: 从某个局面出发，按当前策略走下去，期望能拿到多少回报。它回答“现在这个局面值多少钱”。注意它依赖于策略——同一个局面，好策略能挣得更多。

这两个量对应后面所有算法的两条主线：**直接调整策略**，还是**先学会评估局面再据此行动**。第 2 节会把这条分岔画成一张图。

关于 π_{θ} 有一句要说在前面：**后面三个算法换的都不是它**。REINFORCE、A2C、PPO 用的是同一个网络结构、同一份 61 进 14 出的接口，换的只是“拿到一批数据之后，用什么公式去调它的参数”。到第 7 节让它学会各种花样时，连参数怎么调都不换了，换的是奖励表。

1.7 训练数据是它自己跑出来的

有一个词后面会反复出现：**rollout**。它指的是“让当前策略在环境里连续跑一段，把这一段经历记下来”。在强化学习里，rollout **就是训练数据本身**，没有别的来源。这带来两个直接后果：

- **数据要现产**。每一轮训练之前都得先跑一段，所以仿真速度直接决定训练速度。这也是为什么这一讲从头到尾在仿真里做，而且要一次开几千个环境同时跑。
- **数据会过期**。用旧策略跑出来的数据，对新策略未必还适用。这件事有多严重、能不能忽略，是第 2 节要讲的一个重要划分。

1.8 这一讲要走的路线

贯穿全讲的基础任务只有一个：让这只鸭子按速度指令在平地上行走。

选它当入门载体有三个理由：它不需要任何示教数据（1.2 节说的第一处困难就绕开了）；“跟上速度、保持直立”这个分数很好写；而且它在仿真里可以大规模并行试错，单卡就能复现。

方法上走一条**三级阶梯**，每一级都是一个完整的算法，在**同一个任务、同一份预算**下跑出来对照：

- **v1 REINFORCE**：最朴素的策略梯度。能走起来，但**站不住**。评测里 6400 个环境步摔了 40 次。
- **v2 A2C**：请一个“评分员”来当参照物。换来的是同样 6400 个环境步里**没有一次摔倒终止**，分并没有更高。
- **v3 PPO**：给每次更新加上护栏。同一次评测里它的动作幅度**更小**、平均奖励**更高**。

这三行各自对应第 4、5、6 节末尾的一次实测，**每一行的重点都不是“分涨了多少”**。4.6 节会说清为什么“每步平均奖励”这个指标对“频繁摔倒”几乎是瞎的。

为了让对照读得清楚，三个版本对齐了能对齐的部分：**同一个环境、同一份 61 进 14 出的接口、同一个网络结构、同一份环境交互步数（6000 迭代 $\times$ 4096 环境 $\times$ 24 步）**，评测也走同一个协议（最终权重、确定性动作、16 个环境各 400 步）。

没有对齐的正是算法本身：v2 多了 critic 与 GAE，v3 还在同一批数据上多训几轮。所以 v3 的环境交互步数与前两档相同，而它的**梯度步数与算力更高**——这不是没控住的变量，这就是三级阶梯要讲的那三处改动。**版本之间的差异（diff）就是这一讲的知识点**——读懂三个文件的区别，就读懂了从 REINFORCE 到 PPO 的演进。

讲完 PPO，第 7 节复用同一个训练入口去学别的动作：轮滑、原地旋转、鼻尖取物、前滚翻。那时候动的全在环境这一侧——奖励表、指令、课程表，或者脚下的那双脚。

1.9 术语对照表

后面各节会反复用到这些词。中英文与符号一并列出，读英文文献时对得上。

中文	English	符号	在这个任务里是什么
智能体	agent	—	那个 61 进 14 出的神经网络
环境	environment	—	机器人身体 + 地面 + 重力 + 指令生成器
状态	state	x_t	环境的完整状态，含策略看不到的量（摩擦、电压）
观测	observation	o_t	策略实际拿到的 61 个数：48 维本体感受 + 13 维指令
特权观测	privileged observation	c_t	只在训练时给 critic 的那一份（第 5 节）
动作	action	a_t	14 个关节的目标角度偏移（弧度）
一步	timestep	t	20 毫秒（50 赫兹控制周期）
奖励	reward	r_t	一张 16 项的加权表，每步算一次
回报	return	R_t	奖励的折扣累加
折扣因子	discount factor	γ	0.99
回合	episode	—	从站立开始，到摔倒（终止）或时间到（截断）
终止 / 截断	termination / truncation	—	摔了 / 到点了，两者不能一样对待
轨迹	trajectory	τ	一个回合里的 (o, a, r) 序列
策略	policy	π_θ	动作概率分布，训练时采样、评测时取均值
价值函数	value function	$V_\phi(c_t)$	当前局面按现行策略还能挣多少
优势	advantage	A_t	这个动作比参照值好多少（4.2 节引入，5.2 节把参照值换成 $V(c)$ ）
探索	exploration	σ	动作分布的标准差，可训练
熵	entropy	$\mathcal{H}$	动作分布有多宽；高斯下是 σ 的单调函数（4.5 节）
概率比值	probability ratio	ρ_t	新旧策略在同一个动作上的概率之比（6.2 节；原论文写作 $r_t(\theta)$ ）
轨迹采集	rollout	—	让策略在环境里跑一段并记下来

2 强化学习方法分类

强化学习的算法出了名的多而杂，名字一大把，很容易读着读着就不知道自己在哪。这一节不求全，只建立一张**不会混淆的地图**：用三个问题把方法分开，每个问题都对应一个真实的工程后果。

这一节不出现公式。

2.1 第一个划分：学不学环境模型

第一个问题是：要不要让机器人先学会“环境是怎么运作的”？

- **有模型** (model-based)：用一个转移模型来推演——给定当前状态和动作，预测下一个状态会是什么。这个模型可以是学出来的，也可以直接由物理规律或仿真器给出。有了它就能在“脑内”推演：不用真的动，先想几步再决定。
- **无模型** (model-free)：不显式用环境模型去推演，直接学“该怎么动”。环境是个黑盒，只管往里送动作、接住奖励——它照样要用「观测、动作、奖励、下一观测」这些交互数据。

工程上的取舍。有模型的**常常**更省真实交互——同一份经验可以在脑内反复推演；但这不是保证：学出来的模型会带偏差，推演与规划本身也要算力。无模型的结构更直接，代价是通常需要更多交互数据。

这一讲全程走无模型。因为我们在仿真里，试错成本低到可以一次开几千个环境，样本效率不是瓶颈；而这一讲要讲的是三种策略优化方法的差别。这是针对眼下这个处境的取舍，不是说有模型那条路不行。

2.2 第二个划分：学什么——基于价值还是基于策略

第二个问题是：网络输出的到底是什么？

- **基于价值** (value-based)：网络学的是“每个动作值多少分”。用的时候挑分最高的那个动作。经典代表是 Q-learning 和 DQN。
- **基于策略** (policy-based)：网络直接学“该做什么动作”，输出一个动作或一个动作分布。经典代表就是策略梯度那一家。

这个划分在我们这个任务上不是偏好问题，而是可行性问题。

基于价值的方法要“挑分最高的动作”，这在动作是**离散**的时候很自然——比如上下左右四个选择，四个都算一遍取最大。但这只鸭子的动作是 **14 个连续的关节角度**，组合是连续无穷的，DQN 那种“逐个动作枚举取最大”的做法直接失效。

要补一句：这不等于价值方法在连续动作上没戏。DDPG、TD3、SAC 都是面向连续动作的 Q 函数方法——它们的办法是**再学一个网络来输出那个“最优动作”**，而不是枚举。只是那样一来就同时有了策略网络和价值网络，落到下一小节那个 Actor-Critic 的框子里去了。

所以这一讲走基于策略这一支。不是因为它更先进，而是因为在 14 维连续动作上它最直接：网络输出的就是要下发的那组角度。

2.3 Actor-Critic：横跨两类的混合架构

上面两类不是非此即彼。实际最常用的是把它们缝在一起，叫 **Actor-Critic**：

- **Actor**（演员）是策略网络：负责输出动作。这是基于策略的那一半。
- **Critic**（评论员）是价值网络：负责评估局面值多少分。这是基于价值的那一半。

它们的分工是：actor 决定做什么，critic 告诉 actor “你刚才那个决定，比这个局面的平均水平好还是差”。actor 拿这个评价去调整自己。

critic 只在训练时存在。训练完把 actor 单独拿走就能用，critic 扔掉。这带来一个很有用的后果——既然 critic 反正不部署，那就可以让它看到 actor 看不到的信息（仿真器里的“天眼”数据），把分打得更准。第 5 节会具体用到这一点。

本讲的 v2 和 v3 都是 Actor-Critic 架构；v1 只有 actor，恰好用来看清“没有 critic 会怎样”。

2.4 第三个划分：采数据的策略和学的策略是不是同一个

第三个问题最不直观，但它的工程后果最大：**你手上这批数据，是“现在这个策略”跑出来的吗？**

区分它们的是**这套算法的更新公式，成立时是否要求数据来自当前策略**：

- **on-policy**（同策略）：公式里的期望是对**当前策略**的轨迹分布取的。数据一旦来自旧策略，那个期望就对不上了，梯度带偏差。
- **off-policy**（异策略）：算法自带修正机制（重要性采样比、或者干脆学 Q 函数这种不依赖行为策略的目标），所以数据可以来自别的策略，包括很久以前的自己。

工程后果差得很远。on-policy 的数据只对当前这一版策略有效，通常采一批、更新完就丢（PPO 会在同一批上多走几个 epoch，靠 clip 把“旧了一点”的偏差压住——所以“用一次就扔”是常见做法而不是定义）；off-policy 能把经验存进池子反复抽用，样本效率高得多，代价是要处理“用旧数据学新策略”带来的偏差，训练稳定性更难对付。

本讲三级阶梯全是 **on-policy**。理由和 2.2 节一样：仿真里数据便宜，我们宁可多跑几轮换一个更稳的训练过程。

2.5 把常见算法放上这张地图

三个划分交叉起来，常见算法各自的位置就清楚了：

算法	学环境模型	学什么	数据来源
Q-learning · DQN	不学	价值	off-policy
REINFORCE	不学	策略	on-policy
A2C · A3C	不学	策略 + 价值	on-policy
TRPO · PPO	不学	策略 + 价值	on-policy
DDPG · TD3 · SAC	不学	策略 + 价值	off-policy
Dreamer 一类	学	策略 + 价值	二者皆有

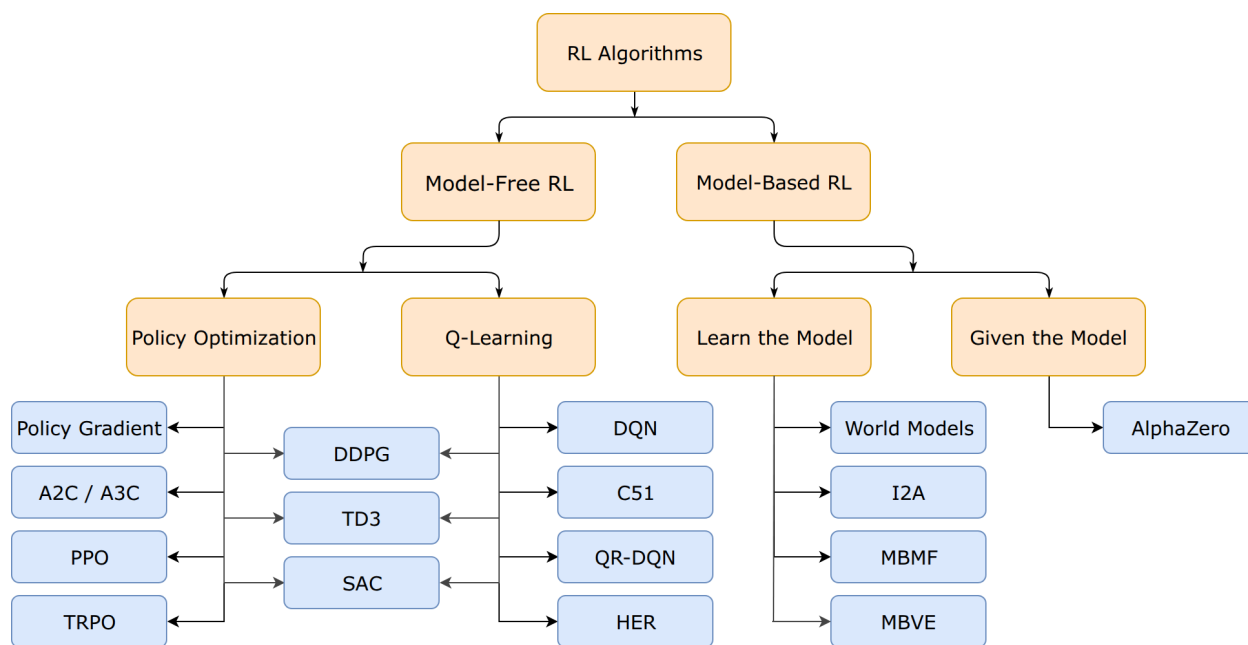


图 9: 方法分类地图（取自 OpenAI Spinning Up）。它画出了本节前两个划分：先分「学不学环境模型」，无模型那一支再分「基于策略」与「基于价值」。本讲走的是左下角那条——无模型·基于策略，图里的 Policy Gradient → A2C/A3C → PPO 正好是第 4-6 节的三级阶梯。第三个划分（on-policy / off-policy）不在这张图上，见 2.4 节。

本讲的路线是：无模型·基于策略 (Actor-Critic) ·on-policy。在这条线上，从最朴素的 REINFORCE 出发，一路走到 PPO。

地图上其余的格子不是被否定了，只是不在这一讲的范围里。

2.6 本节小结

三个问题，三句话：

- **要不要先学环境？** 不学——仿真里试错便宜，不值得多引入一个会学错的东西。
- **输出价值还是动作？** 输出动作——14 个连续关节角没法“逐个算过来取最大”。
- **数据能不能重复用？** 这一讲不重复用——宁可多跑，换训练过程更稳。

三个“这一讲的选择”都不是因为它们最先进，而是因为它们最匹配眼下的处境：连续高维动作、仿真里可以海量试错、想要一个尽量少踩坑的入门路径。

3 认识这只小鸭子：从硬件到仿真

前两章讲的都是通用概念，从这一节起落到具体这台机器上。

要说清一件事：**介绍硬件不是为了教你怎么装一台机器人**，而是为了让你知道**被控对象是什么**。1.4 节那 61 个数字和 14 个数字凭什么是那样，答案全在这一节里。

3.1 它是什么：800 克、25 厘米、两条腿



图 10: 25 厘米高是什么概念：它站在桌上，人伸手过去就能把它整个握住。机身上外露的黑色方块就是舵机，脚是橙色的塑料底板。（厂商照片，Pollen Robotics）

高 × 宽	25 厘米 × 14 厘米
重量	不到 800 克
关节	15 个舵机（Dynamixel XL330 一类的微型数字舵机）
计算	Rockchip RK3566，带 AI 加速器，1 GB 内存 / 32 GB 存储
传感	前置摄像头、8×8 飞行时间深度矩阵、 两个 惯性测量单元（躯干与头各一）
其他	麦克风、扬声器、NFC、Wi-Fi、蓝牙
电池	可换的 NP-F550 相机电池（2600 毫安时），续航约 1 小时
控制频率	50 赫兹

它比一瓶矿泉水重一点，放在桌上大约一个笔记本电脑屏幕那么高。整机与全套软件开源，定价 399 美元一档，这个价位是它值得作为教学载体的原因之一——摔坏一台的代价，和摔坏一台人形机器人不在一个量级。

这个尺度决定了很多事。它轻，所以摔了不容易坏，可以放心让它反复摔；它小，所以关节能出的力矩也小（约 1 牛·米，相当于在 10 厘米长的杠杆末端挂 1 公斤）。后面 3.6 节会说明，正是这个“力矩

很小”的事实，让我们不能把它的舵机当成一个理想的位置伺服来仿真。

3.2 骨架与关节：14 个舵机怎么分布，第 15 个在哪

它的运动来自 14 个舵机，分布已经在 1.4 节列过：每条腿 5 个，颈与头 4 个。开篇那张四只鸭子的照片可以对着数——舵机和连杆全是外露的。



图 11: 一款模块化小鸭子的爆炸图，可以看清这类机器人是怎么装起来的：头部上下两片外壳夹住眼睛模块，颈部旋转舵机连到颈支架；上身外壳里是主控板与电池模块，底下压着中框架；腕、大腿、小腿、脚踝的舵机一节接一节串下去，最后是脚部底板与防滑垫。**舵机就是那些黑色方块，连杆是它们之间的金属件——全部外露，不做外壳。**图里画的是每条腿四节（腕 / 大腿 / 小腿 / 脚踝）加颈部一节；本篇仿真的这只把腕做成三个轴、头上另有三个，所以数到 14。

需要补一句的是真机上有 15 个舵机——多出来那个驱动一张能开合的嘴。那不是装饰：真机靠这张嘴

叼起小物件，官方演示里就有这一项。但仿真模型里嘴是一个**被动连杆**，不参与运动控制，因为本篇要学的动作（走路、起身、踢球、轮滑）都用不到它。

这是“仿真只建与任务有关的自由度”的第一个具体例子。多建一个自由度就多一份物理计算、多一维需要学习的动作，而它对我们要学的动作没有贡献。

站立姿态（也就是每个回合的起点）大致是：髋俯仰约 26 度、踝约 26 度、膝接近伸直、颈与头各约 20 度。这个姿态是调过的——躯干比几何中心略微前移，让重心正好落在踝关节轴上方；早先的版本重心偏后，起身策略学出来的动作是把头往前甩当配重。

3.3 大脑与神经：板子、总线、传感器、电池

- **计算板**：瑞芯微 RK3566，带一个 AI 加速器，1 GB 内存、32 GB 存储。巴掌大小，功耗低到能靠一块相机电池供一小时。
- **舵机总线**：所有舵机挂在一条串行总线上，板子通过它读关节角、下发目标角度。注意是一条总线：15 个舵机轮流通信，这是后面“指令延迟”那一项的物理来源。
- **两个惯性测量单元**：一个在躯干、一个在头部。给出姿态（四元数）、角速度、加速度。1.4 节那 48 维本体感受里，“它现在是正着的还是歪的”就来自这里。为什么要两个：头是可以独立转动的，只靠躯干那一个测不出头的朝向。
- **摄像头与 8×8 深度矩阵**：头部有一个前置摄像头和一个飞行时间深度传感器（输出 8×8 的距离矩阵，相当于一个极低分辨率的激光雷达）。本篇的策略完全不用它们——走路、起身这些动作靠本体感受就够，视觉与深度是另一类任务的入口。
- **电池**：一块可换的 NP-F550 相机电池，标称 7.2 伏、2600 毫安时，续航约一小时。仿真里把工作电压建在 6.5 到 8.2 伏之间——满电到亏电的实际区间。这个区间 3.7 节要用到。

3.4 它平时怎么被驱动：50 赫兹的控制环

真机上跑着一组各管一件事的常驻服务，它们用本地套接字上的一套 JSON-RPC 通信：控制环与电机总线、配置与配网、固件更新、蓝牙、手柄输入、摄像头与音频、深度传感器。其中只有控制环那一个服务碰机器人本体，而且**它的循环从不等待任何别的服务**——跨服务读数据一律读缓存里的最新值，不做同步调用。这条约束是为了保住 50 赫兹的确定性：一个卡住的摄像头服务不能把控制环拖慢。

一个控制周期（20 毫秒）里发生的事，顺序是固定的：

1. 板子从总线上读回 14 个关节的角度与角速度，从惯性单元读回姿态；
2. 这些数字连同当前指令拼成那 61 维观测；
3. 策略算一次，输出 14 个目标角度；
4. 目标角度下发到总线，各舵机的位置环去追它；
5. 20 毫秒后回到第 1 步。

把这个数字变具体很有必要，因为后面几处都要用它：回合长度是多少步、指令延迟是几个周期、“动作变化太快”这一项在惩罚什么，全都以这 20 毫秒为单位。

3.5 仿真里的这只鸭子：三个模型，各有各的用处

仿真里的机器人是一份 MJCF 文件（MuJoCo 的模型格式），从 CAD 导出、带上质量与惯量。这里有三份，不是一份：

模型	特点	谁在用
走路模型	躯干与头的碰撞体被剥掉	走路、轮滑
全碰撞模型	身体能真的躺在地上	起身、坐站、取物、踢球
轮滑模型	脚下多四个被动轮	轮滑、旋转

为什么要剥掉碰撞体？因为走路任务里“摔倒”只需要被判定为终止，不需要精确模拟躯干砸在地上怎么弹。少一批碰撞体就少一批接触检测，训练明显更快。而起身任务恰恰相反——它必须从“躺在地上”开始，身体与地面的接触是任务的核心，那批碰撞体一个都不能少。

同一个道理再说一次：**仿真只建与任务有关的东西**。这是把算力花在有用的地方。

3.6 舵机不是理想的 PD：BAM 执行器模型

这一节解释一个容易被跳过、但在这个尺度上非做不可的事。

最省事的仿真做法是：把舵机当成一个理想的位置伺服——你给目标角度，它就以某个刚度去追，力矩要多少有多少（顶多设一个上限）。大机器人上这个近似常常够用。

在这只鸭子上它不够用，因为它的力矩本来就只有约 1 牛·米。这意味着：

- 电池电压掉一点，能出的力矩就少一截；
- 关节转得快的时候，反电动势会把可用力矩进一步压下去；
- 减速箱里的摩擦不是一个常数，它随负载变化。

三件事叠起来，理想 PD 与真实舵机的差距不是“精度问题”，而是**量级问题**。用理想 PD 训出来的步态，可能是靠一个它其实拿不出来的力矩来源站住的——那样的策略在真实力矩下站不住。

所以这里用的是一个把舵机建到**电压控制律**层面的模型：输入是目标角度，中间算过电压、反电动势、库仑摩擦与随负载变化的摩擦，输出才是力矩。

3.7 每个回合都不完全一样：域随机化

如果每次训练面对的都是同一台机器、同一块地面，策略很容易学出一个**只对这一组参数成立的巧合**：它精确地利用了某个摩擦系数、某个电压，换一点就不成立。

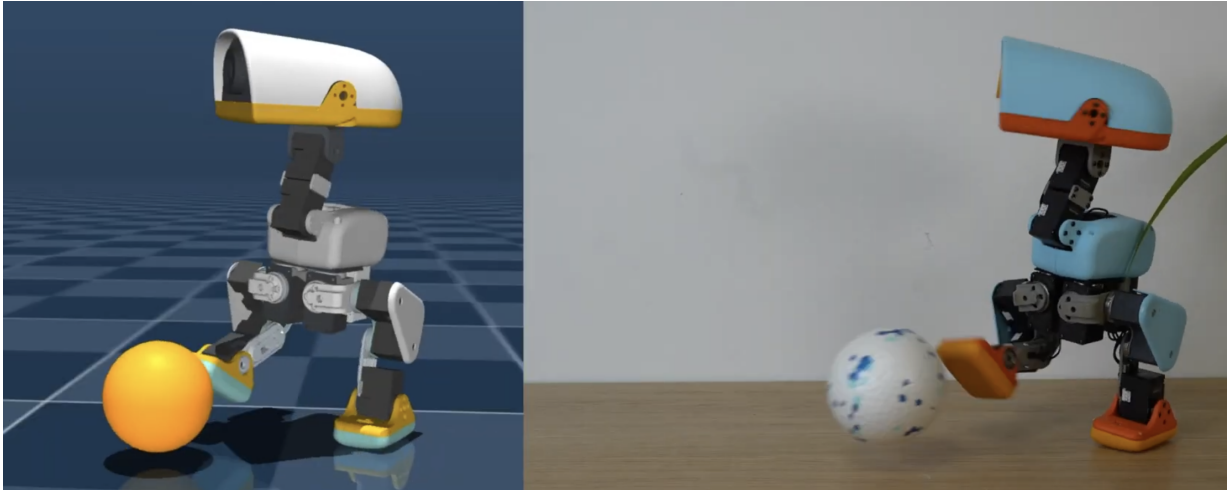


图 12: 同一个踢球动作，左边是仿真、右边是真机。这一对图说明本篇为什么要在执行器上下功夫：两边的姿态、支撑腿的位置、摆腿的时机都能对上，而对得上的前提是仿真里的舵机不是一个理想的位置伺服。

对策叫**域随机化** (domain randomization)：每个回合开始时，把那些“本来就不确定”的量重新摇一遍。这里摇的是：

量	为什么摇它
电池电压	电量从满到亏，工作电压在 6.5–8.2 伏之间变
负载下的电压跌落	多个舵机同时出力时，总线电压会被拉低
指令延迟（3–6 个周期）	总线通信与固件处理都要时间，不是零延迟
关节摩擦	装配公差、磨损、温度都会改变它
质量与惯量、质心位置	电池位置、线束走向都会影响
惯性单元的安装误差	传感器不可能装得绝对正
编码器标定偏差	每个关节的零位都有一点偏差
训练中随机推一把	让它学会被扰动之后自己找回来

于是策略见到的不是一台固定的机器，而是一**族略有差别的机器**。它学出来的东西必须对整族都成立，这就排除了那些只对一组参数成立的巧合。

上游还建了一套齿隙模型（每个舵机串一个 ± 1 度的空程），用来对付减速箱里的间隙。本篇不用它。

3.8 mjlab：一次开几千只鸭子的练功房

强化学习靠海量试错吃饭。一个环境一步一步跑，一天也训不出来。这里用的仿真框架叫 **mjlab**，它做两件事：

一是把 **MuJoCo** 搬上 **GPU** 并行。它用 GPU 版的 **MuJoCo** (**MuJoCo Warp**)，同时跑几千个互相独

立的物理世界——每个世界里一只鸭子。注意这句话的重点：不是“一个场地里站着几千只鸭子”，而是“几千份各自独立的世界，每份里一只”。它们之间既看不见也碰不到，各有自己的速度指令、自己的回合计数、自己的重置时机。

二是把环境拆成一组管理器。场景、观测、奖励、终止条件、指令生成、域随机化、课程表，每一样都是一个可插拔的配置项。1.5 节那张奖励表，在代码里就是奖励管理器的一份配置。这个设计有个重要后果，第 7 节会讲：**奖励与课程表都在环境这一侧，换掉训练算法它们照样生效**。

并行开多少个合适？实测过一条吞吐曲线（只算仿真、不含学习）：

并行环境数	2048	4096	8192	16384
每秒仿真步	15.98	15.00	10.70	5.49
每墙钟秒采到的环境步（百万）	0.033	0.061	0.088	0.090

读法：2048 到 4096 几乎免费（数据翻倍而每秒步数只掉 6%），说明那时候瓶颈还在“启动一次计算”的开销上；4096 到 8192 净赚约 1.45 倍；再往上完全打平——吞吐到顶了。**甜点在 8192**。

顺带一个反直觉的观察：**显存从来不是约束**。16384 个环境时张量占用不到 1 GB。瓶颈在核函数启动开销与 CPU 侧的编排，不在算力也不在显存——所以换一张显存更大的卡并不会更快。

3.9 代码地图：一节讲义对一个模块

这一讲从下一节开始，每一个一级标题都对应代码仓库里的一个文件。

讲义	代码	内容
第 4 节	<code>code/train_v1_reinforce.py</code>	走路 · REINFORCE
第 5 节	<code>code/train_v2_a2c.py</code>	走路 · A2C
第 6 节	<code>code/train_v3_ppo.py</code>	走路 · PPO
第 7 节	同一个 <code>train_v3_ppo.py</code> ，只改 TASK 一行	换成别的动作

另外三个共用件：

- `code/env.py` —— 把 mujlab 的任务包成薄接口。**这是整个目录里唯一与机器人有关的文件**，顶部一行 TASK 决定跑哪个任务。
- `code/model.py` —— 三个版本共用的 ActorCritic 网络，以及算回报与梯度权重的那几个函数。三个算法用**同一个网络类**，所以版本之间的差距不会被网络结构的差异污染。
- `code/rollout.py` —— 按统一口径评测并录像。

三份训练脚本都是同一个 PyTorch Lightning 结构：一个在线采样的 Dataset、一个 LightningDataModule、一个负责算 loss 的 LightningModule，入口统一是 `trainer.fit(model, data)`。要调的超参就写在第一次用到它的地方，没有命令行参数层。

装环境与跑起来：

```
# 按 pyproject 装依赖，不用 pip
uv sync
# 冒烟：环境构得起来、观测维度对得上
uv run pytest tests/ -q
# 训一个策略，权重落到 RL_DUCK_CKPT_ROOT
uv run python code/train_v3_ppo.py
```

建议横向对照着读那三份脚本。它们的差异就是接下来三节的全部内容。

4 v1 REINFORCE：先自己试，试好了就多做

4.1 直觉：没有标准答案，怎么知道该往哪调

监督学习里，网络参数怎么调是清楚的：有标准答案，算出误差，往误差变小的方向走。

强化学习没有标准答案。它手上只有一堆“我做了这个，后来拿到这么多分”的记录。问题变成了：**只知道分数，怎么知道参数该往哪调？**

答案朴素得让人意外：

凡是最后拿分多的那些动作，让它们以后更容易被选中；拿分少的，让它们更不容易。

就这一句。它不需要知道“正确动作”是什么，也不需要知道环境是怎么运作的。它只需要能对同一个局面尝试不同的动作，然后看结果。

打个比方。你在一个陌生城市找一家好吃的面馆，没有地图也没有点评。你能做的就是随便试：这家好吃，下次更可能再来；那家难吃，下次绕开。试的次数足够多，你的选择分布就会自己向好吃的那些倾斜。你从头到尾没有得到过“哪家最好吃”这个标准答案，但你的行为改善了。

这就是**策略梯度**（policy gradient）的全部直觉。

4.2 好动作加概率，坏动作减概率

把上面那句话落到网络上。

先看清网络到底输出什么。它不是给出 14 个角度，而是给出 14 个**中心值**（1.6 节那张图里的 $\mu_\theta(s)$ ），真正下发的动作是在这些中心附近采出来的一组数。所以“让某个动作更容易被选中”，落到参数上就是一件很具体的事：**把中心朝那个动作挪一点**。

举个数：某一步左膝的中心是 0.30 弧度，采样采出了 0.36，鸭子照 0.36 迈了这一步。如果这一段的回报是正的，就把左膝的中心从 0.30 往 0.36 那边挪一点点——比如挪到 0.31。下次再遇到类似的局

面，采样就更容易再落在 0.36 附近。回报越大，挪得越多。**回报是负的就朝反方向挪**，把中心从 0.36 推开，下次更不容易再这么迈。

十四个关节同时这么挪，每一条采到的数据 (o_t, a_t, R_t) 都贡献一次这样的挪动，一整批数据的挪动叠起来，就是这一轮的更新。

（对高斯分布来说这不是类比，而是字面成立的：“提高 a_t 概率”的梯度方向正比于 $(a_t - \mu)/\sigma^2$ —— 从中心指向这次那个动作。所以“沿着让概率变大的方向走”和“把中心朝这个动作挪”是同一件事的两种说法。）

这里有一个必须处理的问题：**回报的绝对值没有意义，只有相对高低有意义。**

假设某个任务里所有动作的回报都在 100 到 110 之间。按上面的规则，每一个动作都会被大幅提高概率（因为 R_t 都是很大的正数），好动作和坏动作都在涨，只是涨得多少不同——这等于什么都没学到，而且梯度还特别大，训练会炸。

所以实践中要**减掉一个参照值**。v1 用的是最朴素的做法：**减掉这一批数据的平均回报**。减完之后“比平均好的加概率、比平均差的减概率”，方向就对了；再除以这批数据的标准差把尺度归一化，梯度大小也稳住。

两点要说清。一是**减完只保证这批优势的均值为零，不保证正负各占一半**——回报分布是偏的（少数几步拿到很高的回报），零点两侧的个数就不相等。二是**基线是压低方差的工具，不是策略梯度成立的前提**：不减基线的 REINFORCE 照样是个有效的估计，只是方差通常大得多。只要基线不依赖当前这个动作，减掉它就不改变梯度的期望——这也是下一节能把常数换成 $V(c_t)$ 的依据。

这里出现了两个后面一直要用的词。被减掉的那个参照值叫**基线**（baseline, b ）；减完得到的那个差——“这个动作比参照值好多少”——叫**优势**（advantage, $A_t = R_t - b$ ）。v1 的代码里那个变量就叫 advantages。

v1 的基线是一个常数：整批的平均回报。第 5 节要做的事，就是把这个常数换成一个随局面变化的、学出来的函数——优势这个词不变，变的是拿什么当参照。

4.3 要写成式子的话

上面那套直觉有严格的表述，叫**策略梯度定理**。结论直接用，不推导：

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | o_t) \cdot R_t \right]$$

左边 $J(\theta)$ 是“按这个策略走，期望能拿多少回报”——正是我们想最大化的东西。右边说：它的梯度可以直接从采样数据里估出来，不需要知道环境的任何内部结构。

怎么读这个式子：

- $\nabla_{\theta} \log \pi_{\theta}(a_t | o_t)$ 是“让这个动作概率变大”的方向，也就是 4.2 节那个“把中心朝它挪”；
- R_t （减掉基线之后就是优势 A_t ）是这个方向该被放大多少倍（负数就反向）；
- 求和与期望的意思是：把采到的所有 (o_t, a_t) 都这样处理，然后平均。

加上基线之后， R_t 换成 $R_t - b$ ，其中 b 是基线。可以证明：只要基线不依赖于动作，减掉它不改变梯度的期望，但能显著降低估计的方差。这就是为什么可以放心减。

跳过这一节不影响读懂后面。要记住的只有一句：梯度 = “提高这个动作概率的方向” × “这个动作比参照值好多少”。

4.4 一轮训练长什么样：采一批、算回报、更新一次

v1 的一次迭代分三步：

第一步，采样。用当前策略在环境里跑一段，记录每一步的观测、动作、奖励。本篇的配置是 4096 个环境同时跑、每个环境跑 24 步，所以一次迭代拿到 $4096 \times 24 = 98304$ 条数据。

第二步，算回报。对每一步算出它的折扣回报 R_t 。做法是从后往前累加：

$$R_t = r_t + \gamma R_{t+1}$$

倒着走是因为第 t 步的回报要用到第 $t+1$ 步的结果。遇到回合结束就把递推截断——不能把下一个回合的奖励算进上一个回合。

这里的 R_t 是 24 步截断回报，不是完整回合的回报。一段 rollout 只有 24 步，而走路的回合是 1000 步——递推走到这一段的末尾就停了，第 24 步之后那些奖励被当成了零。而 $\gamma^{24} \approx 0.79$ ，那部分并不小。

严格的 REINFORCE 要采到回合真正结束再算回报。v1 没有这么做，也没有 critic 去给这一下段的尾巴补一个估值（那是第 5 节 v2 才有的东西），所以 v1 的回报估计带有截断偏差——它是“基于 24 步截断回报的有偏近似”，不是教科书上那个无偏的 Monte Carlo 估计。本篇仍按惯例叫它 REINFORCE，指的是它的更新规则。

第三步，更新一次。用这批数据算一次梯度，走一步，然后把数据扔掉。下一轮重新采。这是 2.4 节说的 on-policy：数据由当前策略产生，策略一更新它就过期了。（“过期”不等于“只能用一次”——只要策略变化还受控，同一批数据在一个短窗口内可以反复训几轮，第 6 节的 PPO 就是这么做的。）

还有一处 v1 没有区分的東西：回合是怎么结束的。1.3 节讲过两种结束方式——摔倒是终止（terminated），后面确实没有价值可言；时间到是截断（truncated），机器人本来还能继续挣分。价值估计上这两种该分开处理：

1. 真正终止：不使用下一状态的价值；
2. 时间截断：应当用重置前那一帧的观测算一个价值，把尾巴补上；

3. 回报或 GAE 的递推不能跨过重置边界；
4. rollout 自然走完但回合还没结束时，同样要额外算一个 `next_value` 补尾项。

v1 的代码里只有一个合并后的 `done`s，四条里只做到了第 3 条——它既没有 critic 可用来补尾巴，也没去看回合是怎么结束的。第 5 节起两件都补上了：v2 与 v3 都算 `next_value` 补尾项，也都读 `info["time_outs"]` 把超时那一步单独加上 $\gamma V(c_t)$ 。所以”终止与截断分开”这件事，是 v1 到 v2 那一步顺带修掉的，只是当时没单独说——它藏在”换了个参照值”这个改动里。

一次迭代 = 采一段 + 更新一次。本篇跑 6000 次迭代。

4.5 代码走读：v1 版训练循环

代码在 `code/train_v1_reinforce.py`。它是标准的 PyTorch Lightning 结构，三个类各管一件事：

DuckRolloutDataset（在线采样）。它的 `sample_rollout()` 就是 4.4 节的第一、二步：

```
# 攒够一批再更新，逐步更新方差太大
for step in range(self.num_steps_per_env):
    # 采样阶段不建图，省显存
    with torch.no_grad():
        actions, _log_probs, _values, _means, _stds = self.model.act(obs,
        ↪ critic_obs)
    # 4096 个环境同时走一步
    next_obs, next_critic_obs, rewards, done, _info = env.step(actions)
    # 观测、动作、奖励、done 都要存，回报要靠它们倒着算
    ...
# done 处截断，别把下个回合算进来
returns = reward_to_go(reward_steps, done_steps, self.gamma)
# v1 的基线只有整批均值这个常数，没有 critic
advantages = returns - returns.mean()
# 归一化，免得批与批之间步长忽大忽小
advantages = advantages / (advantages.std() + 1.0e-8)
```

注意 `torch.no_grad()`：采样阶段不需要梯度，那是更新阶段的事。也注意最后两行——那就是 4.2 节说的“减掉整批均值再除以标准差”，v1 的基线只有这两行。

DuckData（把环境交给 Trainer）。它长期持有环境对象，因为仿真状态要跨迭代连续推进，不能每轮新建。它的 `train_dataloader()` 每次被调用都重建一次数据集——重建就意味着重新采样。

这里有个必须显式打开的开关。Lightning 默认只在训练开头建一次 `dataloader` 并一直复用，光定义 `train_dataloader()` 保证不了每个 epoch 重新采样。所以 Trainer 上写着：

```
# 每个 epoch 重建一次 dataloader ——重建即重新采样，on-policy 靠这一行成立
reload_dataloaders_every_n_epochs=1,
```

漏了它，训练照样跑、照样不报错，只是每轮都在用开头那一批旧数据 —— on-policy 就名存实亡了。三个训练脚本都设了它。

DuckLightningReinforce（算 loss）。整个算法的核心就是 training_step 里这几行：

```
# 策略输出的是分布，不是一个动作
distribution = self.model.action_distribution(batch["obs"])
# 14 个关节独立，log 概率相加
log_probs = distribution.log_prob(batch["actions"]).sum(dim=-1)
# 熵 = 这个分布有多宽；宽 = 还在到处试
entropy = distribution.entropy().sum(dim=-1)
# 优势为正就把这个动作的概率推高
policy_loss = -(log_probs * batch["advantages"]).mean()
# 单独拿出来，好调它的权重
entropy_loss = entropy.mean()
# 减号：鼓励熵，不是惩罚熵
loss = policy_loss - self.entropy_coef * entropy_loss
```

第四行就是 4.3 节那个式子——前面加负号是因为优化器做的是**最小化**，而我们要最大化回报。

最后一行多了一项**熵奖励**（entropy bonus）。

熵（entropy, $\mathcal{H}$ ）在这里只有一个含义：**这个动作分布有多宽**。1.6 节那张图上，策略给的是均值 μ 加标准差 σ —— 动作就在均值附近按这个宽度采。 σ 大，采出来的动作散得开，同一个局面下这次迈 0.30、下次可能迈 0.42，熵就大； σ 小，每次都紧贴均值，熵就小。**熵大 = 还在到处试**，**熵小 = 已经拿定主意**。（高斯分布的熵有闭式： $\mathcal{H} = \frac{1}{2} \log(2\pi e \sigma^2)$ ，所以它就是 σ 的单调函数 —— 谈熵和谈“探索幅度”是同一件事。）

为什么要专门奖励它：策略梯度只会把中心朝“这次拿分多的动作”挪，而每挪一次， σ 收一点也能让分数更稳 —— 于是它有一个短视的偏好：**早早把 σ 收到很小，锁在当前这个还不错的动作上**。锁住之后就再也采不到别的动作了，而“别的动作也许更好”这件事，不采到就永远不知道。所以在 loss 里减去一个熵项（减号 = 奖励高熵），相当于给“保持分布宽一点”付一点钱，把这个短视偏好压住。系数取 0.01 —— 很小，它只是不让分布过早塌掉，不是鼓励一直乱试。

还有一处值得注意：

```
# v1 没有 critic，只优化 actor 和那个标准差
params = list(self.model.actor.parameters()) + [self.model.std]
# 标准差是可学的：探索幅度自己收敛
self.optimizer = torch.optim.Adam(params, lr=self.learning_rate)
```

只有 actor 和动作标准差进优化器。网络类里虽然带着 critic，但 REINFORCE 用不到它——刻意让它闲置，这样三个版本才能共用同一个网络类，对照时不掺入结构差异。

4.6 实验：它真能走过来，但走得难看

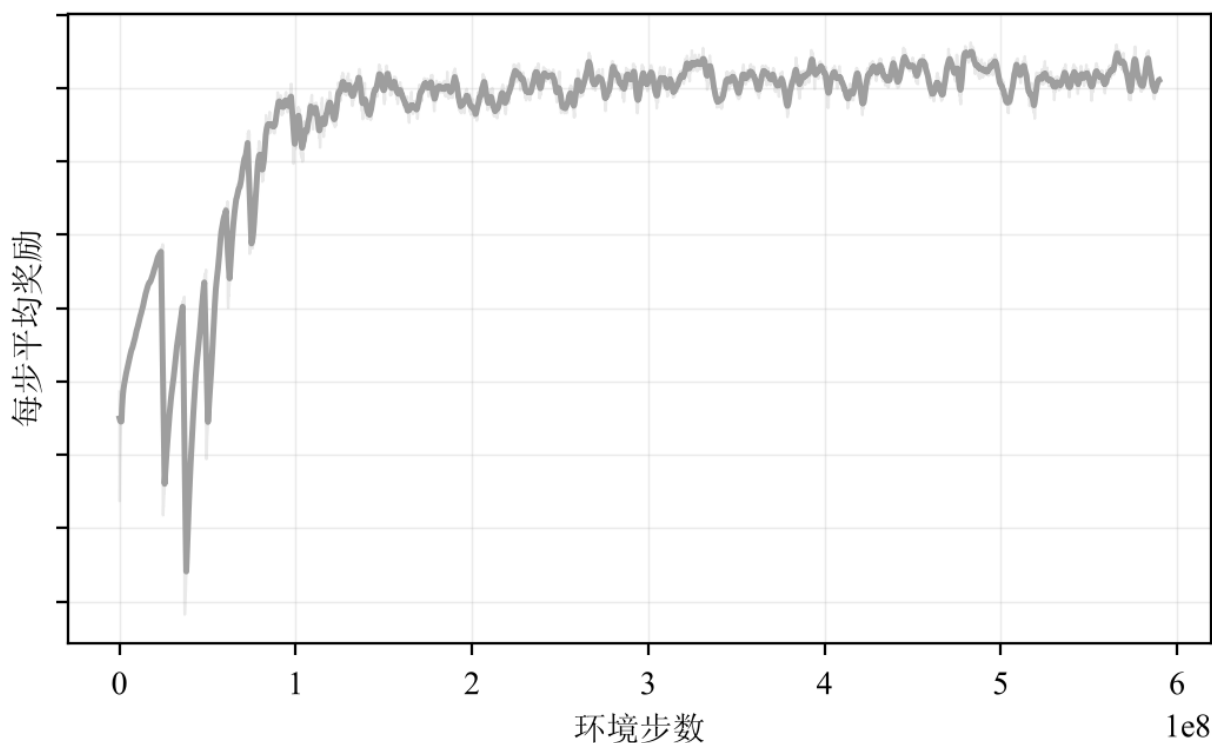


图 13: v1 REINFORCE 的学习曲线。浅色是逐迭代原始值，深色是滑动平均。它确实在学，但爬到 0.06 附近就走平了，而且是彻底走平，不是放慢。

它真的走起来了。前一千多次迭代曲线一路向上，鸭子从原地抽搐变成能往前挪。这件事本身值得停一下：我们没有给过任何一个“正确动作”的示范，奖励表里也没有一句话描述“怎么走”，只有“跟上速度指令”“别摔”“别抖”这些评分标准。14 个舵机的协调是它自己试出来的。

但它很早就见顶了。从大约第 1300 次迭代起，曲线在 0.060 附近彻底走平，不动了，不是变慢。后面几千次迭代、上亿个环境步，换来的提升接近于零。

按统一口径评测（最终权重、确定性动作、16 个环境各跑 400 步）：

指标	v1 REINFORCE
评测平均奖励	0.127
摔倒终止	40 次 （6400 个环境步里）
动作幅度均值	0.248

先看中间那一行，它是这一节真正的结果：**40 次摔倒**。

16 个环境各跑 400 步，一共 6400 个环境步，摔了 40 次，平均每个环境摔两次半。也就是说这个策略走不了几秒就会倒，倒了重置，再走几秒再倒。

而它的“评测平均奖励”是 0.127，比下一节那个不摔倒的 **v2 (0.122)** 还高一点点。这件事值得停下来想清楚，因为它是整篇里最容易骗人的一处：每次摔倒之后环境把鸭子重置成一个漂亮的站姿，而站姿附近是最容易拿分的地方。于是“**频繁摔倒 + 频繁重置**”这个策略在“**每步平均奖励**”这个指标上并不难看。一个只看平均奖励的人会得出“v1 和 v2 差不多”的结论，而看视频的人一眼就分得出来。

“走得难看”就是从这十二帧上看出来的：步子很小、几乎不抬脚，靠上身晃动把身体往前带。

4.7 毛病在哪：回报的噪声淹没了学习信号

v1 能学，但学得慢，而且很早就见顶。原因不在“梯度算错了”——策略梯度定理是对的，v1 的估计也是无偏的。问题在**方差**。

想一下 R_t 是什么：从第 t 步往后所有奖励的折扣累加。这个数取决于**这一整条轨迹后面发生的所有事**——包括与第 t 步那个动作毫无关系的事。

举个具体的：鸭子在第 10 步做了一个很好的迈步动作，但第 40 步因为一个和它无关的扰动摔倒了，于是第 10 步的回报也很低。算法据此得出结论：“第 10 步那个动作不好，压低它的概率”。**它把功劳和罪责算错了人**。

这个现象叫**信用分配**（credit assignment）问题。用整条轨迹的回报当权重，噪声非常大：同一个状态、同一个动作，两次采样得到的 R_t 可能差很远。减掉整批均值只解决了“尺度”，没解决“噪声”——那个常数基线对所有状态是同一个值，而不同状态的合理参照本来就该不同。站得稳的局面和快摔倒的局面，“平均水平”显然不一样。

需要一个**随状态变化的参照值**。而“某个状态值多少分”这件事，1.6 节已经定义过了——它就是价值函数 $V(c)$ 。下一节把那个常数换成它。

4.8 本节小结

- 策略梯度的全部直觉是一句话：**拿分多的动作加概率，拿分少的减概率**。

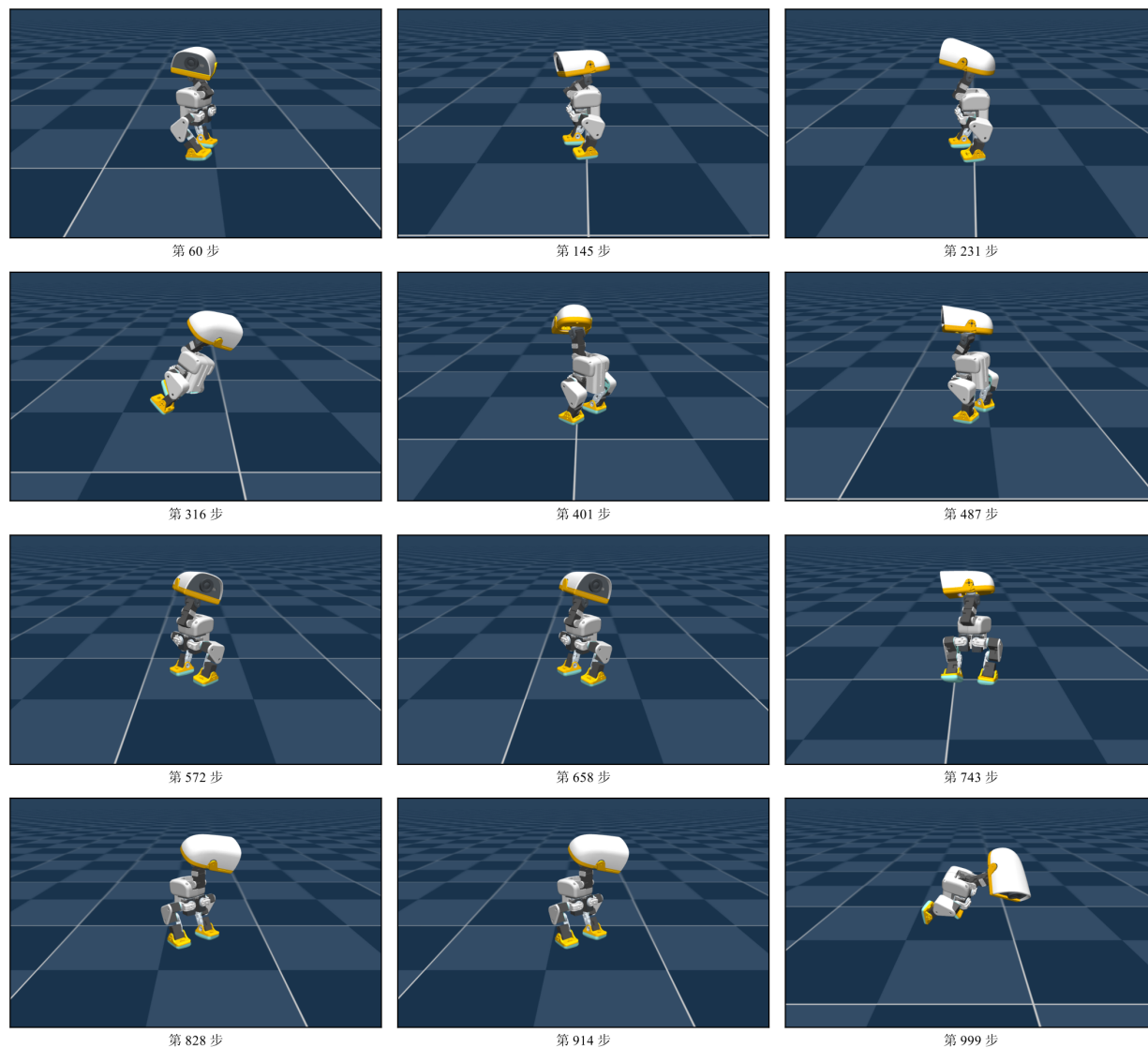


图 14: v1 REINFORCE 最终策略的一整个回合，十二帧覆盖 1000 步（相邻帧 85 步）。这一回合跑满 1000 步、没有触发摔倒终止。这十二帧里没有一帧是干净的迈步姿态——双脚都在身体正下方；第 316 步整个躯干前倾到头快贴地，第 401 步又立回来，末帧（第 999 步）又是一次深前倾。帧与帧之间头的朝向差别最大。（离散取样看不到两帧之间的短暂动作，「有没有迈步」要看交付/视频/单只-走路-v1REINFORCE.mp4。）

- 通常要减掉一个参照值（基线）来**压低方差**；只要基线不依赖当前动作，减掉它不改变梯度的期望。它是降方差的工具，不是策略梯度成立的前提。v1 的基线是整批均值，再除以批内标准差做尺度归一化。
- 一次迭代 = 采一段数据 + 更新一次 + 把数据扔掉（on-policy）。
- 它的毛病是方差：用整条轨迹的回报当权重，会把功劳和罪责算错人。

5 v2 A2C：请一位评分员

5.1 直觉：跟「平均水平」比，不是跟零比

v1 的问题是参照值太粗——所有状态共用一个常数。

改进的直觉很自然：**每个局面应该有自己的参照值**。

还是找面馆那个比方。你在一条美食街上，随便一家都不错；在另一条街上，能吃饱就算走运。同样一顿“七分满意”的饭，在美食街上是失望，在另一条街上是惊喜。如果你用同一个标准（比如“及格线六分”）去评判两条街上的所有选择，你学到的东西就是错的。

正确的做法是：**拿这次的结果，跟“这个局面本来预期能拿多少”去比**。比预期好就加概率，比预期差就减。这个“本来预期能拿多少”就是价值函数 $V(c)$ 。

优势还是 4.2 节那个优势——“这个动作比参照值好多少”。**换掉的只是参照值**：v1 拿整批均值这个常数当参照，v2 拿 $V(c_t)$ ——一个随局面变化的、学出来的参照。

5.2 评分员是什么：价值与优势

引入第二个网络——**critic**，它学的就是 $V(c)$ ：给它一个局面，它预测“按当前策略走下去，期望还能拿多少回报”。

于是原来的权重 $R_t - \bar{R}$ （实际回报减整批均值）换成

$$A_t = R_t - V(c_t)$$

实际拿到的，减去这个局面本来预期能拿到的。这就是优势。

优势的符号有非常直白的含义：

- $A_t > 0$ ：这个动作比“在这个局面下的平均表现”更好 → 加概率。
- $A_t < 0$ ：比平均更差 → 减概率。

对比 v1：v1 是“跟全局平均比”，v2 是“跟本局面的预期比”。后者把“局面本身好不好”这个因素**扣掉了**，剩下的才是“这个动作贡献了多少”。这正是 4.7 节要的东西。

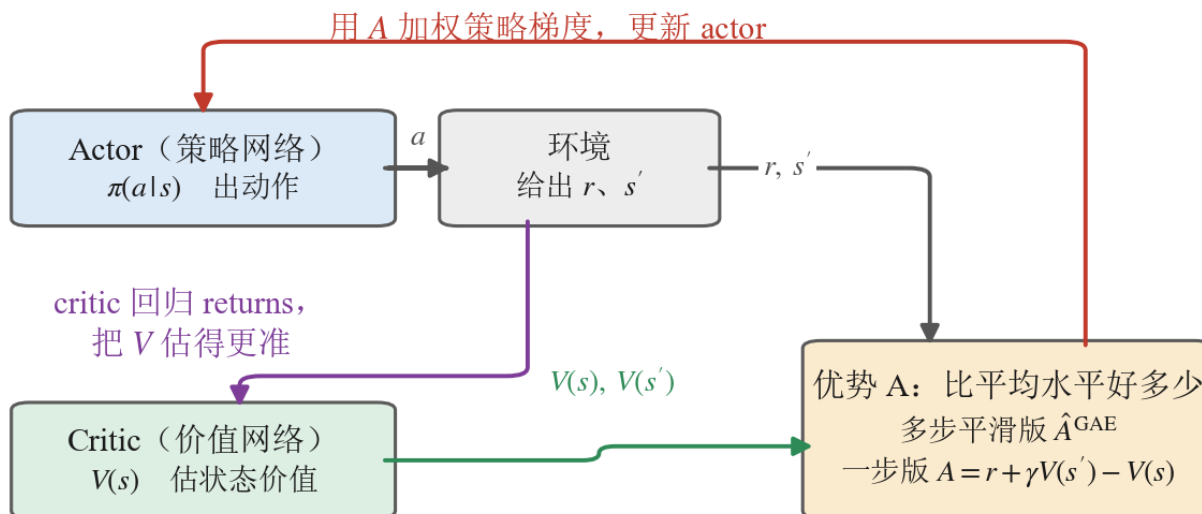


图 15: actor 与 critic 各自在做什么。critic 回归 returns 把 V 估准，两个 V 相减得到优势 A （一步版 $A = r + \gamma V(c_{t+1}) - V(c_t)$ ；本篇用的是多步平滑的版本，5.3 节讲），再拿 A 去加权策略梯度更新 actor。

Actor 和 critic 一起训练，各有各的目标：

- **actor** 的目标：让优势大的动作概率变大（和 v1 一样，只是权重换成了 A_t ）。
- **critic** 的目标：让自己的预测更准，也就是让 $V(c_t)$ 贴近实际回报。这是一个普通的回归问题，用均方误差。

这就是 **Actor-Critic**，2.3 节在地图上标过的位置。

一个可以白拿的好处。 critic 只在训练时存在（2.3 节说过）。既然它不部署，那就可以让它看到 actor 看不到的东西——比如仿真器里精确的躯干线速度、没有加编码器偏置的真实关节角、脚底的接触力。这些“天眼”信息让它估得更准，而 actor 依然只看那 61 个真机拿得到的数字。本篇的代码就是这么做的：走路这个任务上 actor 61 维、critic 76 维，**多出 15 维**。那 15 维全是真机上拿不到、仿真器里却是现成的量——基座真实线速度 3、双脚离地高度 2、双脚腾空时间 2、双脚是否触地 2、接触力 6。（actor 那 61 维所有任务通用；critic 多看多少则按任务而定，第 7 节那几个动作是 74 到 78 维。）这几项恰好都是“评价这一步走得好不好”最需要的信息，而它们又都是装不上传感器的（真机没有全局速度计，脚底也没有力传感器）。critic 只在训练时存在，所以它可以用这些量；actor 要被搬到真机上，所以一个都不能用。

5.3 要写成式子的话

优势的定義可以更一般： $A(o, a) = Q(o, a) - V(c)$ ，其中 $Q(o, a)$ 是“在观测 o 下做动作 a ，之后按策略走能拿多少”。 $A_t = R_t - V(c_t)$ 是它的一个采样估计。

实践中不直接用 $R_t - V(c_t)$ ，而是用一个叫 **GAE**（广义优势估计）的东西。它的动机是这样的：

- 用 $R_t - V(c_t)$: R_t 是真实采到的，所以**无偏**，但它包含整条轨迹的噪声，**方差大**。
- 只往前看一步 $r_t + \gamma V(c_{t+1}) - V(c_t)$: **方差小**，但严重依赖 critic 估得准不准，critic 不准就有**偏**。

GAE 用一个参数 $\lambda \in [0, 1]$ 在这两端之间连续插值：

$$\delta_t = r_t + \gamma V(c_{t+1}) - V(c_t), \quad A_t^{\text{GAE}} = \sum_{k \geq 0} (\gamma \lambda)^k \delta_{t+k}$$

$\lambda = 0$ 退化成只看一步（方差低，但依赖 critic 准不准，**偏差大**）， $\lambda = 1$ 退化成完整回报（**偏差小**、**方差大**）。本篇取 $\lambda = 0.95$ —— 这个值靠的是**低偏差、高方差那一端**，是实践里常用的折中。

实现上同样是从后往前**递推**，一次扫描就能算完整段：

$$A_t = \delta_t + \gamma \lambda \cdot A_{t+1}$$

跳过这一节不影响读懂后面。要记住的是：**优势 = 实际减预期**；GAE 是一个在“准”和“稳”之间调节的旋钮，本篇调在**偏稳那一侧**。

5.4 代码走读：v1 到 v2 改了什么

code/train_v2_a2c.py 与 v1 的结构完全一样，差异集中在两处。

关键变化一：优势不再用常数基线，改用 critic 估值 + GAE。

v1 是：

```
# v1: 回报照旧这样算
returns = reward_to_go(reward_steps, done_steps, self.gamma)
# v1: 基线是一个与状态无关的常数
advantages = returns - returns.mean()
```

v2 换成：

```
# (T, N, 1)    critic 给每一步一个" 这个局面值多少分"
values = ...
# (N, 1)       最后一步之后再估一次，用来接上被截断的尾巴
next_value = ...
# 多步平滑，   调偏差与方差
advantages, returns = compute_gae(rewards, dones, values, next_value, gamma, lam)
# 同样要归一化
advantages = (advantages - advantages.mean()) / (advantages.std() + 1.0e-8)
```

`compute_gae` 就是 5.3 节那个从后往前的递推，它同时返回优势和 critic 要回归的目标（`returns = advantages + values`）。

上面那儿行里，超时自举去哪了。`compute_gae` 里只有一个 `dones` 掩码，而 4.4 节说过真终止与超时截断要分开处理——两者不矛盾，因为自举项不在掩码里，在它上一行的奖励里：

```
# 超时那一步：把  $\gamma V$  加回奖励，抵掉掩码即将清零的尾项
rewards = rewards + self.gamma * values.squeeze(-1) *
    ↪ info["time_outs"].float()
```

于是 `compute_gae` 里那个掩码只负责递推不跨回合边界这一件事，该接的价值尾项在进函数之前就已经加在奖励上了。上游 `rs_l_rl` 就是这么分的。只看 `compute_gae` 会以为超时被当成真终止处理了；两行要一起读。

关键变化二：loss 多了一项 `value_loss`。

`v1` 的 loss 只有策略项和熵项；`v2` 加上 critic 的回归项：

```
# critic 的活：把回报回归准
value_loss = torch.square(values - batch["returns"]).mean()
# 策略项：把中心朝优势大的动作挪（要最小化，所以前面是负号，写在 policy_loss 里）
# 价值项：让 critic 的估值贴住实际回报，系数 1.0
# 熵项：奖励分布保持宽一点，所以是减，系数 0.01
loss = policy_loss + self.value_loss_coef * value_loss - self.entropy_coef *
    ↪ entropy_loss
```

三项各管一件事，加起来一次反传：

项	谁的目标	要它往哪走	系数
<code>policy_loss</code>	actor	优势大的动作，概率变大	1（不另外配）
<code>value_loss</code>	critic	估值贴住实际回报	<code>value_loss_coef = 1.0</code>
<code>entropy_loss</code>	actor 的标准差	分布别过早收窄	<code>entropy_coef = 0.01</code> ，前面是减号

前两项是加、第三项是减，原因只有一个：优化器做的是最小化。前两项本身就是“越小越好”（策略项已经带了负号、价值项是平方误差），而熵是“越大越好”，所以要减。

对应地，优化器现在要管整个网络，不再只管 actor：

```
# v2 起 actor 与 critic 一起优化
self.optimizer = torch.optim.Adam(self.model.parameters(), lr=self.learning_rate)
```

“只改两处”说的是算法上的改动，不是文件级的差异。把两份脚本按函数逐个比过（去掉 docstring 与注释之后比源码）：18 个函数里 5 个逐字相同，8 个内容不同，v1 那份多一个 `reward_to_go`（v2 用 GAE 之后不再需要它）。

那 8 个“内容不同”里，真正承载算法差异的只有 `sample_rollout`（优势怎么算）与 `training_step`（loss 多一项）这两个；其余六个是**连带改动**——`__init__` 多存了 critic 相关的超参、`configure_optimizers` 从“只管 actor”变成“管整个网络”、`main / run_training / setup / train_dataloader` 换了 run 名与算法标签。

这个数字值得说清，因为“只改两处”如果按字面理解，读者一 diff 就会觉得被骗了。**真正的对照保证在别处**：同一个 ActorCritic 网络类、同一份超参、同一个环境与奖励表、同一套评测口径。算法之外的东西一样，才是“只有算法不同”的实义。

5.5 实验：动作幅度打开了，但时不时被推坏

v2 跑满 6000 迭代，按统一口径评测（最终权重、确定性动作、16 环境 × 400 步）：

指标	v1 REINFORCE	v2 A2C
评测平均奖励	0.127	0.122
摔倒终止	40 次	0 次
动作幅度均值	0.248	0.332

这张表只有一行是重点：**摔倒从 40 次变成 0 次**。

平均奖励几乎没变（0.127 → 0.122，甚至略降），但**同样 6400 个环境步里，v1 摔了 40 次、v2 没有出现摔倒终止**。这一步换来的是**站得住**，不是“分更高”。

机理在 4.7 节已经讲完了：常数基线对所有状态用同一个参照值，于是“站得稳的局面”和“快要摔倒的局面”被同等对待，策略学不会区分它们。换成价值函数之后，“这一步比这个局面的平均水平好还是差”这个判断才有意义——而“快摔倒的局面里做了个救回来的动作”正是靠这个判断才能被加强。**状态相关的基线压低优势估计的方差，这是这一步该带来的东西**；上面那两个数是这一次训练、这一份最终权重、这一段评测里量到的。

代价写在“动作幅度均值”那一行：**0.248 → 0.332，动作变猛了三成**。

站得住不是白来的。v2 敢押上更大的动作（这正是“有了可信的参照值才敢下决定”的表现），但它没有任何机制约束“一次别改太多”——所以它的动作幅度一路涨，下一节 v3 会看到，PPO 用更小的动作（0.271）拿到更高的分（0.155）。**v2 的问题是“学过头了”，不是“没学会”**。5.6 节讲这个。

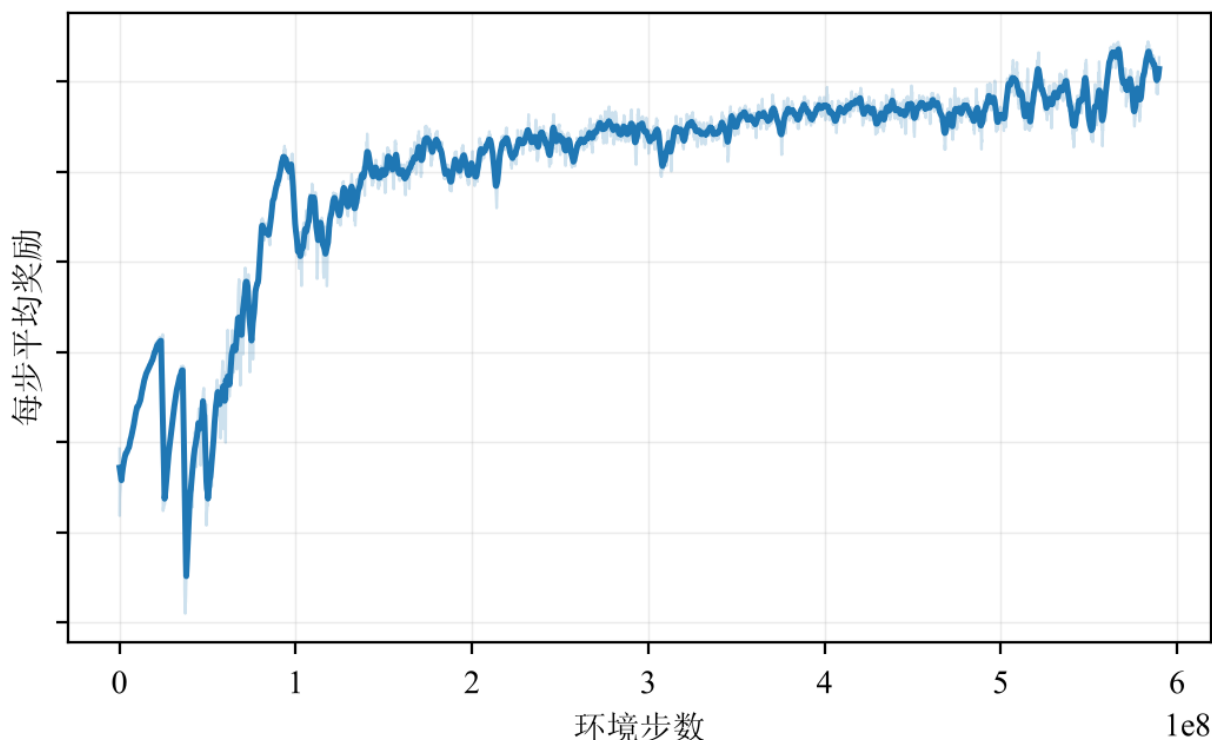


图 16: v2 A2C 的学习曲线。它一路爬到 0.10 以上还没走平，而 v1 在 0.06 就停住了。

5.6 毛病在哪：更新的步子没人管

v2 比 v1 好，但它引入了一个新问题，而且这个问题是**策略梯度**这一类方法共有的。

回到 4.2 节那个规则：把动作分布的中心朝这次的动作挪，挪多远按优势定。问题是：**走多远算合适**？

- 走太小，学得慢。
- 走太大，策略可能一步跨到一个糟糕的区域——而且**回不来**。

为什么回不来？这是 on-policy 特有的陷阱。策略一变，下一轮采到的数据就来自新策略。如果这一步把策略推坏了，下一轮采到的全是坏数据，基于坏数据再更新一次，只会更坏。**它没有一个“退回去”的机制**——监督学习里数据集是固定的，走错一步下次还能纠回来；强化学习里数据是策略自己产的，走错一步连数据都跟着坏掉。

而“合适的步长”没法用一个固定的学习率解决，因为它依赖于当前局面：有时候优势很大，同样的学习率就会造成很大的策略变化；有时候优势很小，同样的学习率几乎什么都没动。

需要一个机制，直接限制“策略变化的幅度”本身，而不是限制学习率。这正是 PPO 做的事。

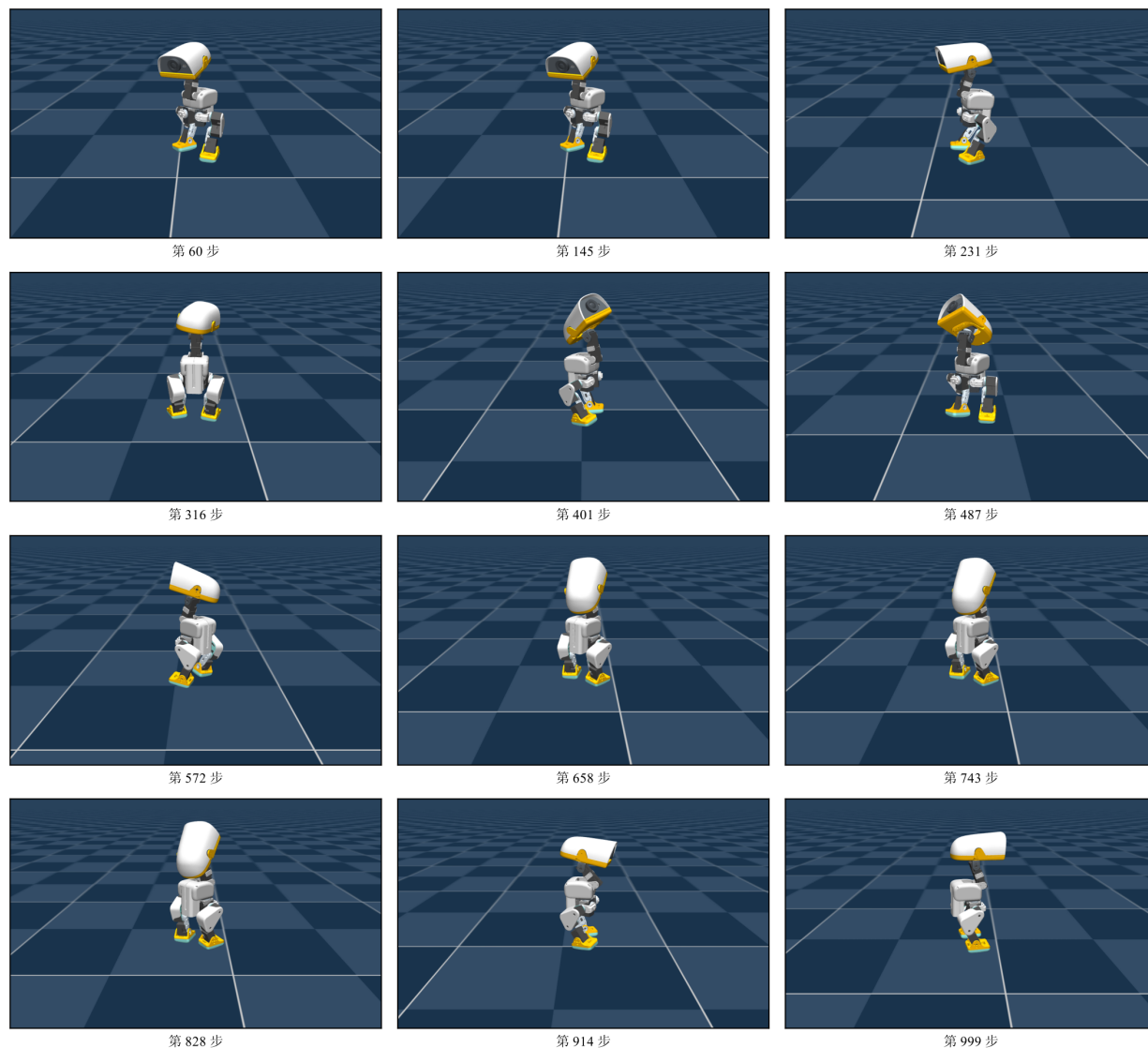


图 17: v2 A2C 最终策略的一整个回合，取样步号与上一张相同。与 v1 相比它在这些取样时刻姿态稳得多：两脚分开站住，第 316 步到第 828 步这七帧几乎一模一样。同样，两帧之间隔着 85 步，短暂的抬脚在这张图上看不见 —— 「站着不动」这个判断要看视频或足底接触记录，不能只凭这十二帧。

5.7 本节小结

- v1 的常数基线换成随状态变化的价值函数，权重从“实际回报”变成“优势 = 实际减预期”。
- 这需要第二个网络 critic。它只在训练时存在，所以可以让它看到 actor 看不到的信息。
- GAE 是在“无偏但噪声大”与“低噪声但依赖 critic”之间的一个旋钮，本篇取 0.95。
- 代码上只改了两处：优势怎么算、loss 多一项。
- 新毛病：更新的步子没人管，一步走坏了就回不来。

6 v3 PPO：给更新的步子加护栏

6.1 直觉：一次别改太多

5.6 节的结论是：要限制“策略变化的幅度”。

那就直接限制它。这个想法有个正式名字叫**信赖域** (trust region)：在当前策略周围划一个“可信区域”，每次更新只允许在这个区域内动。区域内的数据还算适用，出了区域就不可信了。

TRPO 把这个想法写成了一个带约束的优化问题：约束的是**采样分布上的平均 KL**，求解靠局部二阶近似加线搜索。要注意它约束的是那个平均值，**不是对每个状态、每个动作都施加精确的硬上限**。它实现复杂、计算贵。

PPO 出自同一个想法，但便宜得多：不解约束优化，而是把代理目标截断，让更新过大的样本不再提供额外收益。代价是它**不提供真正的信赖域保证**。两者的实际效果都取决于任务、超参与实现；本篇后面只讲 PPO 这一支。

一句话概括 PPO：**如果这次更新想让某个动作的概率变化超过一定比例，就把超出的那部分收益“剪掉”，让它没有继续推的动力。**

6.2 护栏怎么加：概率比值与 clip

怎么衡量“策略变了多少”？用**概率比值**，本篇记作 ρ_t ：

$$\rho_t(\theta) = \frac{\pi_{\theta}(a_t | o_t)}{\pi_{\theta_{\text{old}}}(a_t | o_t)}$$

换个字母是有意的。PPO 原论文把这个比值写成 $r_t(\theta)$ ，而本篇从第 1 章起 r_t 一直是**这一步的奖励**。同一份文档里一个字母担两个意思，读到 $r_t A_t$ 时就得先分辨它是“奖励乘优势”还是“比值乘优势”。所以比值在这里用 ρ ；查原论文时它就是那个 $r_t(\theta)$ 。

分母是**采样时**那个策略给这个动作的概率，分子是**当前**（已经更新过几次的）策略给的概率。比值等于 1 表示没变；大于 1 表示这个动作变得更容易被选；小于 1 表示更不容易。

于是“限制策略变化”就变成“限制这个比值别离 1 太远”。PPO 的做法是：

$$L = \min(\rho_t A_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \cdot A_t)$$

其中 ϵ 本篇取 0.2。**注意它保证的是**什么。clip 只把**这一个样本**的代理目标截断——比值超出 $[0.8, 1.2]$ 之后，这个样本不再提供把策略继续往那个方向推的激励。它**不保证**所有样本的比值都留在区间内，也不保证整批更新之后的策略变化或 KL 散度不越界：多个样本的梯度叠起来，完全可能把某些动作

的比值推到 1.2 以外。所以它是**软约束**，实际训练里还要盯着 KL —— 6.4 节那个学习率自适应就是这个的。

(另外一处用词：连续动作上策略给的是**概率密度**，所以 ρ_t 严格说是**概率密度比**。下文为了顺口仍简称比值。)

这个式子怎么起作用，分两种情况看就清楚了：

- **优势为正**（这是个好动作，想加概率）：比值涨到 $1 + \epsilon$ 之后，clip 把它按住， L 不再随比值增大而增大——**继续推没有收益了**，梯度归零。
- **优势为负**（坏动作，想减概率）：比值降到 $1 - \epsilon$ 之后同理被按住。

而 $\min$ 的作用是保证护栏只在“我们想推的那个方向”上生效：如果策略已经跑到区域外面、而且是朝着更糟的方向，梯度仍然能把它拉回来。

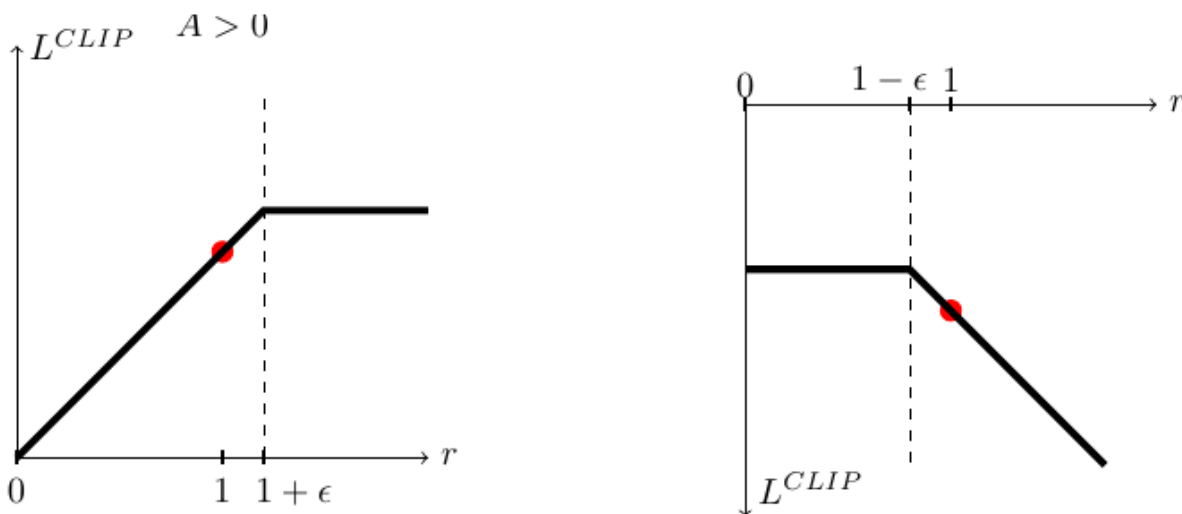


图 18: clip 目标的形状 (PPO 原论文图 1, 横轴那个 r 就是本篇的 ρ_t)。纵轴是单个样本的目标值。左：优势为正时，比值涨过 $1 + \epsilon$ 之后曲线变平 —— 再往那个方向推没有额外收益。右：优势为负时，比值跌破 $1 - \epsilon$ 之后同样变平。红点是比值等于 1（新旧策略相同）的位置。

这就是全部的护栏。没有约束优化，没有二阶导数，就是一个 $\min$ 和一个 clip。

6.3 顺手省数据：一段数据反复训几轮

护栏还带来一个额外的好处，而且是很实在的好处。

$v1$ 和 $v2$ 都是“采一段、更新一次、扔掉”。为什么不能多更新几次？因为更新一次之后策略就变了，这段数据不再来自当前策略——再用就有偏差。

但 PPO 有护栏。既然它能保证策略不会跑太远，那么这段数据在“跑太远之前”都还算适用。于是可以：

- 把一段 rollout 切成几个 minibatch;
- 在这段数据上反复训几轮（本篇 5 轮，每轮 4 个 minibatch）。

同一批数据被用了 5 次而不是 1 次。**样本效率直接提升数倍**，而这正是同样的仿真预算下 PPO 能比 v1/v2 走得好得多的主要原因之一。

注意这不是“变成 off-policy 了”，但理由不是“数据还不够旧”——2.4 节说过，划分看的是行为策略与目标策略的关系。PPO 里每一批数据由一个冻结的行为策略 π_{old} 采出来，在有限几轮内靠概率比修正着用，然后丢掉；它没有一个长期保留的经验池。这才是它仍算 on-policy 的原因。

要承认一件事：**从第二轮起，当前策略已经不等于 π_{old} 了**，这批数据对当前策略已经有偏离——这正是概率比与 clip 要处理的东西。而有限复用本身也不保证训练不发散，所以还要盯 KL（下一节）、必要时提前停这一批。

6.4 学习率自己调：动得太猛就减速

还有一个实用件。 ϵ 限制了单个动作的概率变化，但整体策略变了多少还有一个更直接的度量：**新旧策略之间的 KL 散度**。

PPO 常配一个自适应规则：

- 如果这一轮的 KL 明显超过目标值（本篇目标 0.01），说明动得太猛 → **调小学习率**；
- 如果明显低于目标值，说明太保守 → **调大学习率**；
- 学习率本身夹在一个预设的上下界之间。

这样就不用手画一整条学习率曲线——它跟着 KL 走。**要调的参数并没有消失**：初始学习率、目标 KL、判”明显”的阈值、每次调整的倍率、上下界，这些都还是超参。它也不是 **PPO 的必备组成**——这是本实现的一个选择，PPO 原论文里没有它。

6.5 要写成式子的话

PPO 的完整 loss 有三项：

$$L = \underbrace{-\mathbb{E}[\min(\rho_t A_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)A_t)]}_{\text{策略项（带护栏）}} + \underbrace{c_v \mathbb{E}[(V_\phi(c_t) - R_t)^2]}_{\text{价值回归}} - \underbrace{c_e \mathbb{E}[\mathcal{H}[\pi_\theta]]}_{\text{熵奖励}}$$

三项分别对应：actor 带护栏地往优势大的方向走、critic 回归那个价值目标 R_t 、熵项防止过早收窄。本篇取 $c_v = 1.0$ 、 $c_e = 0.01$ 、 $\epsilon = 0.2$ 。

式子里的 R_t 要说清是什么。它不是走完整个回合算出来的实际回报，而是 5.3 节 GAE 顺手给出的那个目标：**returns = advantages + values**——采样时那一版 critic 的估值，加上 GAE 优势。也

就是说它里面含着 critic 自己的尾部估计（教科书里叫**自举的 λ -return**）。第 4 节 v1 里那个 R_t 才是纯粹从奖励累加出来的，两者同名不同物。

和 v2 的 loss 比，唯一的形状变化就是策略项从 $-\log \pi \cdot A$ 变成了那个 `min(clip(...))`。价值项也跟着做了一次同款裁剪——不让 critic 的估值一次跳得离旧估值太远，默认开着；熵项一字未动。

6.6 代码走读：v2 到 v3 改了什么

`code/train_v3_ppo.py` 与 v2 的差异有三处。

关键变化一：clip——用概率比值把更新夹在信赖域内。

这要求采样时就记下“旧策略”给每个动作的对数概率，因为更新时策略已经变了、补不回来。所以 v3 的采样循环多存了几个量：

```
actions, log_probs, values, means, stds = self.model.act(obs, critic_obs)
# log_probs / means / stds 都要存下来 ——它们是“旧策略”的记录，
# 后面算概率比值与 KL 都要拿它当参照；不存就没法判断" 这一步跨了多远"。
```

更新时：

```
# 新旧策略在同一个动作上的概率比值。旧策略的 log 概率是采样时存下来的
ratio = torch.exp(log_probs - batch["log_probs"])
# 没有限制的那一项
unclipped = ratio * batch["advantages"]
# 把比值夹在 1±
clipped = (torch.clamp(ratio, 1.0 - self.clip_param, 1.0 + self.clip_param)
           * batch["advantages"])
# 取小的那个：越界之后再推没有收益
policy_loss = -torch.min(unclipped, clipped).mean()
```

四行，就是 6.2 节那个式子。

关键变化二：一段 rollout 切成多个 minibatch、反复训几轮。

```
# 数据复用实现在数据集里：一段 rollout 切成 num_mini_batches 个小批、重复吐
↪ num_learning_epochs 轮
def iter_mini_batches(self, rollout):
    batch_size = rollout["actions"].shape[0] * rollout["actions"].shape[1]
    mini_batch_size = batch_size // self.num_mini_batches
    # Lightning 那边一行不用改，多轮复用完全由这个生成器完成
    for _ in range(self.num_learning_epochs):
        ...
```

v1/v2 这里只有一次更新。就是这个双层循环把样本效率提上去的。

关键变化三：按 KL 自适应调学习率。

```
# 算 loss 时顺手拿到这一步的 KL，记下来
loss, policy_loss, value_loss, entropy_loss, kl, mirror =
    ↪ self.loss_for_batch(mini_batch)
self.latest_kl = kl.detach()

# 真正迈步之前才调学习率。调的是 Lightning 传进来的那个 optimizer ——
# 它才是本步会被 step 的那一个
def on_before_optimizer_step(self, optimizer):
    self.latest_lr = adapt_learning_rate_to_kl(optimizer, self.latest_kl,
        ↪ self.desired_kl)
```

`adapt_learning_rate_to_kl` 是一个十几行的小工具：KL 超过目标两倍就把学习率除以 1.5，不到目标一半就乘 1.5，并夹在 10^{-5} 到 10^{-2} 之间。

同样把两份按函数比过：5 个逐字相同、8 个内容不同、v3 多出十个新函数（KL 自适应学习率、minibatch 切分、单批 loss、几个 Lightning 钩子等）。和 5.4 节一样，“**三处**”说的是算法上的改动——承载它的是 `sample_rollout`（多存旧策略的对数概率）与 `training_step`（clip 与多轮复用），其余是把这三件事接进 Lightning 生命周期所需的脚手架。

三处算法改动，就是 v2 与今天这套常用配方之间的全部距离。

6.7 实验：三个算法同台，行走任务收口

三个版本对齐了环境交互与评测：同一个环境、同一个网络结构、4096 个并行环境 × 每轮 24 步 × 6000 次迭代（**环境交互步数相同**；v3 在同一批数据上多训几轮，所以它的梯度步数与算力更高）。评测走同一个协议：各自的最终权重、确定性动作（取分布均值，不掺探索噪声）、16 个环境各跑 400 步。

版本	算法	评测平均奖励	摔倒终止	动作幅度均值	脚离地时间
v1	REINFORCE	0.127	40 次	0.248	——
v2	A2C	0.122	0 次	0.332	0.025 秒
v3	PPO	0.155	0 次	0.271	0.123 秒

这张表要横着读两遍，因为单看任何一列都会读错。

第一遍看“摔倒终止”。同样 6400 个环境步里，v1 摔了 40 次，v2 与 v3 都没有出现摔倒终止。这是 v1 到 v2 那一步换来的东西——而它在“评测平均奖励”那一列**看不出来**：v1 的 0.127 甚至比 v2 的

0.122 高一点，因为每次摔倒后的重置都把鸭子放回一个好拿分的站姿。平均奖励对“频繁摔倒”这件事几乎不敏感。

第二遍看后两列。v2 和 v3 都不摔了，差别在动作质量：v2 用更大的动作（0.332 对 0.271）拿到更低的分（0.122 对 0.155），而它的脚离地时间只有 v3 的五分之一——它在用力蹭地，v3 才是真在走。这是 v2 到 v3 那一步换来的东西。

v1→v2 换的是“站得住”，v2→v3 换的是“走得好”。两步各修一个毛病，而没有任何单一指标能同时说清这两件事。

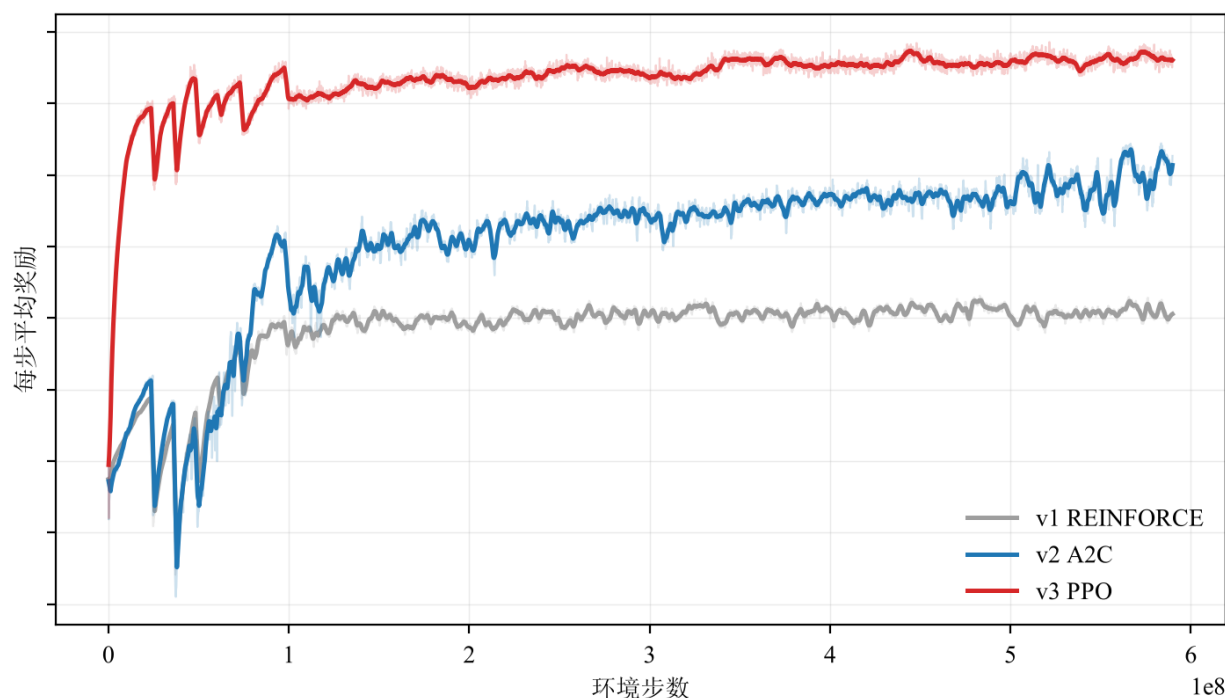


图 19: 同一个任务、同一份预算，三个算法的学习曲线。横轴用环境步数而不是墙钟——墙钟依赖当时跑在哪张卡上、有没有别的任务在抢，换台机器数字就变；环境步数是算法本身消耗的经验量，跨机器可比。浅色是逐迭代原始值，深色是滑动平均，纵轴是训练时记的每步平均奖励。看的是谁早早走平、谁到预算末尾还在爬、谁前期在抖。

这张图比表格说得更清楚：

一，v1 和 v2 在最开始那一段几乎重合。前七千万个环境步里两条线缠在一起——因为那一段两者学的是同一件事（“别摔”），而那件事对基线的质量不敏感。分野出现在之后：v1 在 0.060 附近彻底走平，v2 继续爬到 0.10 以上。基线换成价值函数，换来的是“学得更远”，不是“学得更快”。

二，v3 那条红线上来得又快又高，但它前期在抖。那些锯齿是 clip 与 KL 自适应学习率在起作用的样子：一旦一步走猛了，KL 变大，学习率被压下来，曲线回落一点再爬。护栏是让曲线跌下去还能回来，不是让它变平滑。v2 那条蓝线同样有锯齿，但它的锯齿更深、恢复更慢。

三，v3 在两千多迭代就把主要的分挣完了。之后到 6000 迭代那一段基本是横的。6.3 节说数据复用把

样本效率提上来了，这就是它的样子：同样的经验量，v3 在 v2 还在爬坡的地方已经收口。

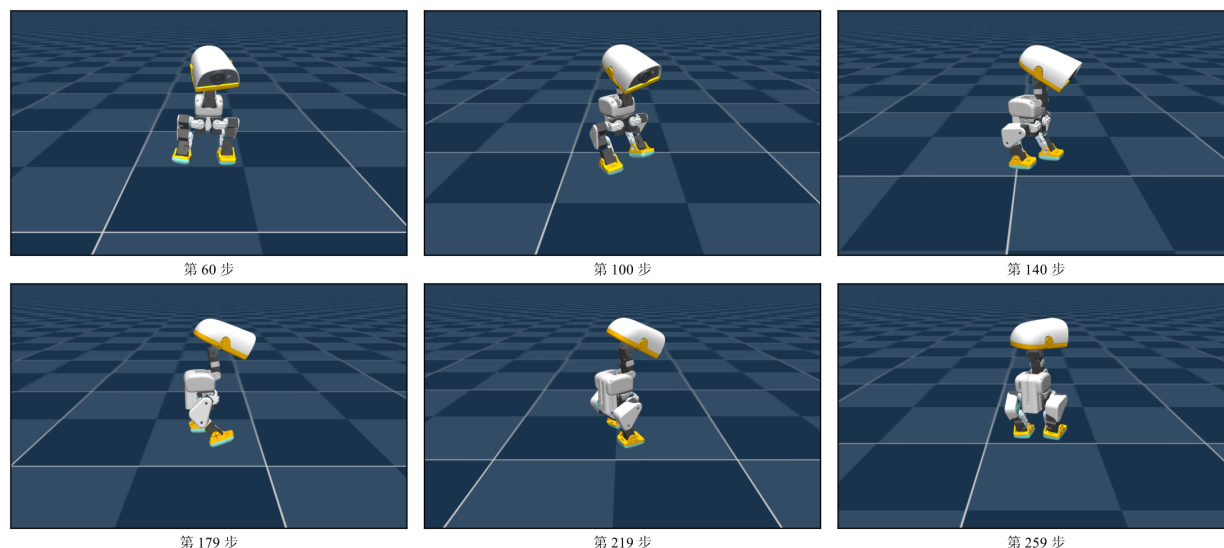


图 20: v3 PPO 最终策略走路的一个片段，六帧（第 60 / 100 / 140 / 179 / 219 / 259 步）。评测里 6400 个环境步零摔倒，脚离地时间 0.123 秒——它是真在迈步，不是蹭地。

6.8 本节小结

- PPO 的核心是一个护栏：用概率密度比衡量策略变了多少，单个样本超出 $\pm 20\%$ 就不再给它继续推的激励。它是软约束，不保证比值或 KL 不越界，所以还配了 KL 自适应学习率。
- 它是信赖域思想的便宜实现——同样的效果，一个 min 加一个 clip 就够，不需要 TRPO 那套约束优化。
- 有了护栏，一段数据就可以反复训几轮（本篇 5 轮 $\times$ 4 个 minibatch），样本效率提升数倍。
- 再配一个按 KL 自适应的学习率，就不用手调它随训练怎么变。
- 代码上与 v2 只差三处。

三级阶梯到此收口。从“拿分多的动作加概率”这一句直觉出发，加一个随状态变化的参照值（v2），再加一个限制更新幅度的护栏（v3），就得到了今天训练机器人步态最常用的那套配方。

下一节做一件不一样的事：**这套配方的训练器不动，切换环境那一侧的配置**，看它还能学出什么。

7 一套 PPO，切换任务配置学不同动作

前三节把一个算法一步步补完，用的一直是同一个任务。这一节反过来：**训练器主体与训练入口不动，切换环境那一侧的配置**，看它能学出什么。

这里说的“不改”只指**复用同一个训练脚本**。不同任务会改奖励，也可能改本体、指令、课程表、重置

与终止条件；前滚翻还额外挂了一项对称性辅助损失。所以有效的优化目标并不完全相同——不变的是那套算法和那份接口。

这一节的写法与前面不同。它不推导、不讲新机制，而是一个任务接一个任务地看：这个动作要什么、奖励表改了哪几项、它最后做出了什么。看点在图。

7.1 谱系总览：一个仓里三十三个任务，算法一行没改

小鸭子的训练仓里注册了 **33 个任务**（python `code/env.py` 打得出来）：**13 个平地主任务**，另外每个主任务多半还有一个“碎石地”孪生版和一个“带齿隙”孪生版（碎石地 10 个、带齿隙 15 个，两者有重叠）。按“改了什么”分成四类：

改的是什么	任务
换奖励	摔倒起身、坐 站、鼻尖取物、踢球、前滚翻
换脚	轮滑跟速度、左右蹬滑行、蹲着滑、滑下坡、从地上站到轮上、原地旋转
换地形	上面多数任务都有一个“碎石地”孪生版
换本体保真度	每个主任务都有一个“带齿隙”孪生版（每个舵机串 ± 1 度空程）

代码上，换任务就是改 `code/env.py` 顶部那一行：

```
# 换成上表里任意一个 task id
_DEFAULT_TASK = "Mjlab-Velocity-Flat-MicroDuck"
```

（批量跑的时候不必改文件——`RL_DUCK_TASK` 这个环境变量按进程覆盖它，一台机器七张卡各跑一个动作，靠的就是这个。）

训练脚本、网络结构、超参、评测口径都不动。本篇从这张表里挑了几个动作训，每个都是同一份 `train_v3_ppo.py`，命令行上只有 `RL_DUCK_TASK` 不同。（唯一的例外在 7.10 节：前滚翻那一档任务注册表里开着对称性增广，训练器查到它就会多挂一项镜像一致性损失。）

下面的编排是：**7.2–7.5 各是一个训出来的动作**，7.6 与 7.7 是从这批任务的奖励表里拆出来的两条通用设计经验，7.8 把前面各节的奖励表并排读出几条规矩。

为什么能这样。 3.8 节提过 `mjlab` 把环境拆成一组管理器：奖励、终止条件、指令生成、域随机化、课程表，每一样都是配置。**这些东西全在环境这一侧，不在算法里。**所以换掉训练算法它们照样生效，换掉任务训练算法也不用改。7.10 节会把这一点讲透。

7.2 脚下装上轮子：轮滑

要什么。 任务目标和走路一样——按速度指令移动、保持直立。换掉的是机器人本体：脚掌换成了四个**被动轮**（不带驱动，只能滚）。本体一换，接触模型、可达的速度范围与终止阈值都跟着变，所以两档不是同一个环境。

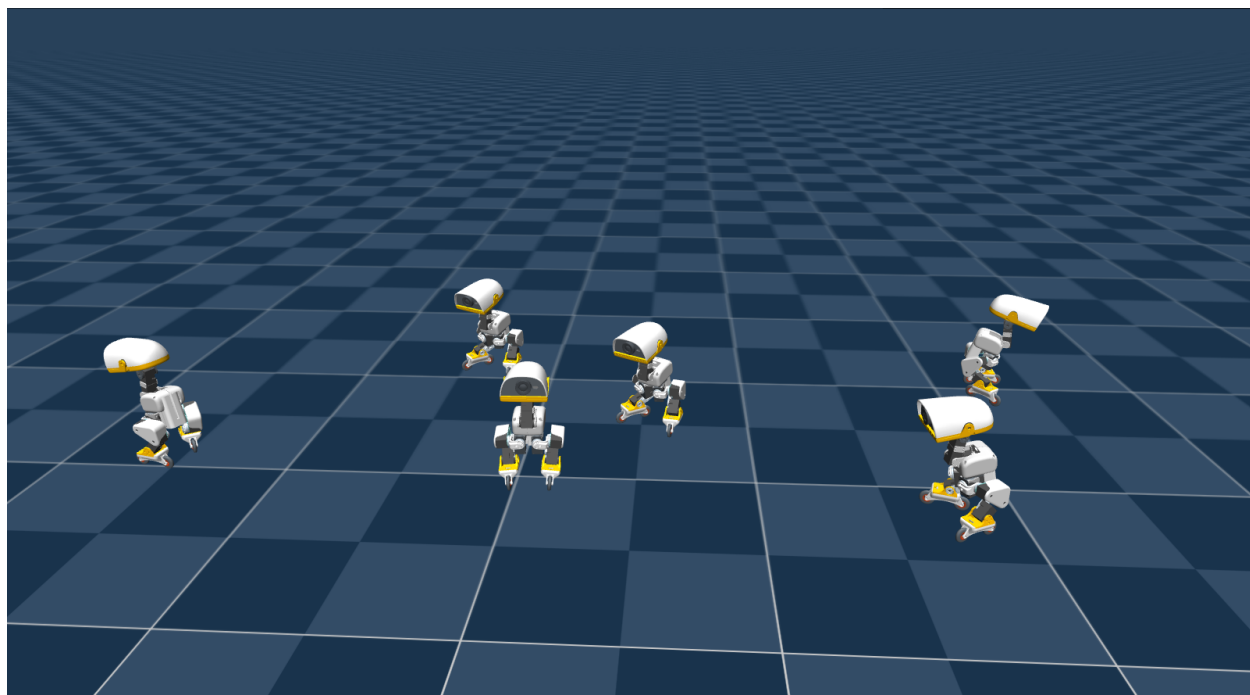


图 21: 脚下换成被动轮之后的策略。六只鸭子同时跑，各自追自己的速度指令——它们都稳稳站在轮子上，靠上身摆动和腿部蹬地产生推力。

奖励改了什么。几乎没改。它用的是同一套速度跟踪奖励。

它学出来的不是走，是滑——靠上身摆动和腿部蹬地产生推力，然后让轮子滚起来。同一套奖励、同一套算法，换掉脚就长出了完全不同的运动模式。

这个任务是这一节最干净的一个例子：**奖励几乎没动，行为却完全变了**。所谓“策略学的是怎么用这个身体拿分”，这就是字面意思——身体一换，怎么拿分的答案也换。

7.3 原地转起来：旋转

要什么。站在轮子上原地快速旋转。

奖励改了什么。主项从“跟上线速度”换成“跟上一个很高的转向角速度”。

它站在轮子上转起来了。这个任务顺带说明一件容易搞错的事：**别用人的尺度去给机器人设限**。一个 25 厘米高的机器人，自然的旋转速度可以到每秒好几弧度——按人形机器人的直觉去加一个“转太快要扣分”的惩罚，会把这个动作直接掐死。上游的做法是：**不限制转速，而是限制冲击与抖动**（竖直加速度、动作变化率），让“转得快”和“转得野”分开。

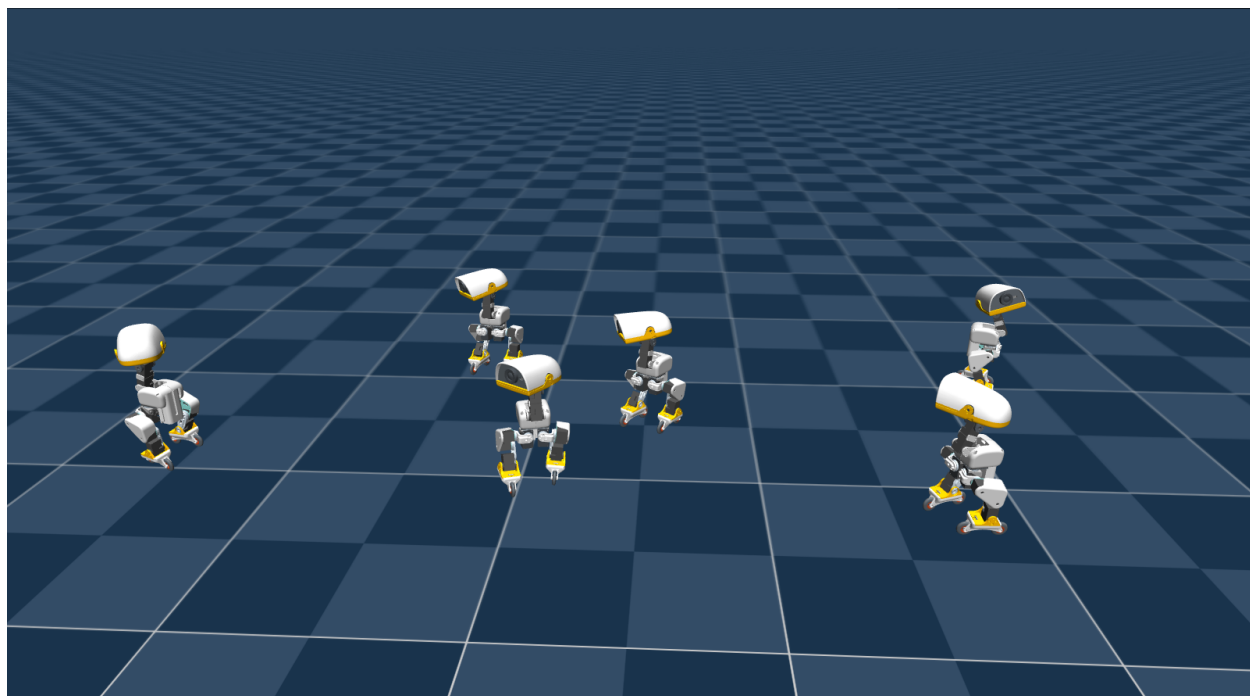


图 22: 原地旋转。六只鸭子在这一帧各自朝着不同方向 —— 它们跑的是同一套策略、只是相位散开了。**单帧只说得清朝向不同，说不了转得多快：转速要看连续画面，或者看角速度曲线（交付/视频/六只-旋转.mp4）。**

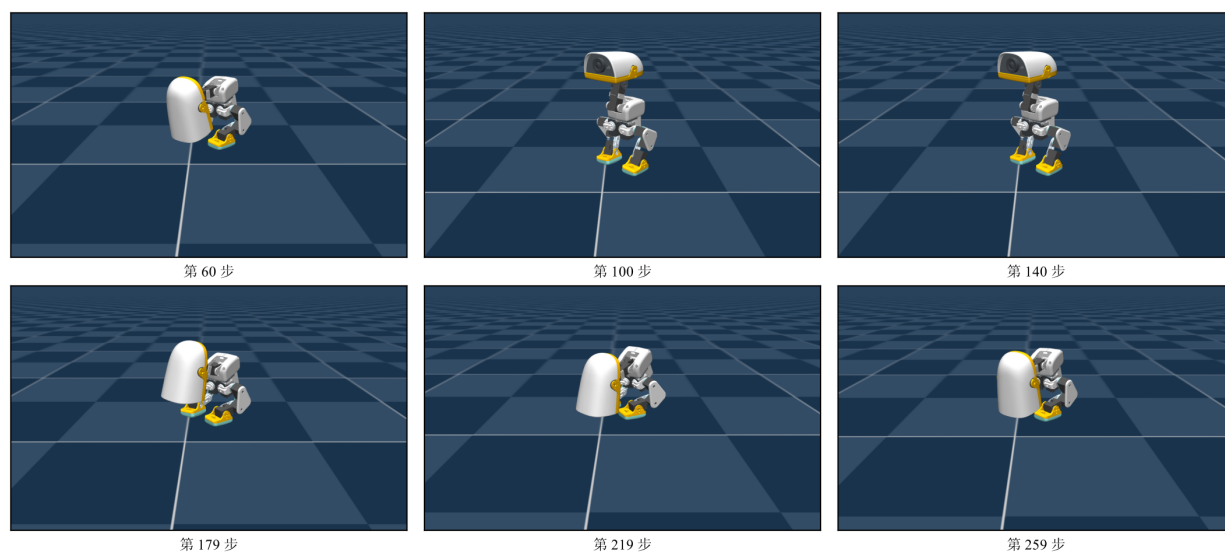


图 23: 嘴尖取物的一个回合，按时间顺序取的六帧（第 60 / 100 / 140 / 179 / 219 / 259 步）。奖励只说了「嘴尖要到那个点」，「先蹲、再前倾、用膝和踝把身体压下去」这套动作是它自己找出来的。

7.4 低头，用嘴尖碰到地上的东西：取物

要什么。从站立姿态蹲下去，用嘴尖碰到地面上一个点，再站回来。

奖励改了什么。这是第一个目标不在“跟指令”而在“到某个空间位置”的任务。奖励表里多了：嘴尖到目标点的距离（高斯型）、蹲下过程中保持平衡、完成后回到站立姿态。

它蹲下去、嘴尖碰到了那个点、再站回来。这个任务的意义在于它换了目标的类型。前面所有任务的目标都是一个速度或姿态，是“整个身体处于什么状态”；这里的目标是末端要够到一个点，身体其余部分怎么配合由策略自己决定。奖励只说“嘴尖要到那儿”，它自己找出了“先蹲、再前倾、用膝和踝把身体压下去”这套动作。

7.5 翻过去再站起来：前滚翻

要什么。从站立开始，向前翻一个跟头，再站回来。

奖励改了什么。走路那套速度跟踪项（跟线速度、跟转向、腾空时间、抬脚高度、摆动腿高度、脚打滑、姿态）七项全删，换成一组围绕“翻过去再站住”的项。主项是翻滚进度（转过了多少角度），权重 +8.0。接着是一组管落地的：落地复合项 +4.0、落地尖峰 +2.0、翻完要直立 +1.5、翻完要站起高度 +1.0、起身速度 +0.75。反向的都很轻：别往侧翻 -0.5、别摊平 -0.5、转太快 -0.1、动作变化率 -0.1。

6000 迭代（它的课程表末档就在 6000，一分不多给），上面那十二帧就是最终权重跑出来的一个回合的前 230 步。

这一节值得细讲，因为它一个动作同时踩到三条规矩，而且每一条都有数。

一、挡动作的正则必须给得很轻（7.8 节规矩（6））。这里有确切的倍数。两个“挡动作”的正则项，同一个名字，在走路和前滚翻两张表里差得很远：

项	走路	前滚翻	倍数
躯干角速度	-0.05	-0.002	轻 25 倍
整机角动量	-0.02	-0.001	轻 20 倍

理由直白得几乎是同义反复：前滚翻这个动作本身就是角动量。按走路那个权重罚下去，等于对动作本身收重税，它永远学不会翻。规矩（6）在第 7 节是一句话，这两行是它的数。

二、“别在坏状态上给正分”是拿一整轮实验换来的（7.8 节规矩（5））。奖励表里有一项专门罚“翻完之后摊着不起来”，它的注释记着上游为这个动作烧掉一轮之后的结论：

带门限的落地奖励让“站起来”比“摊成一堆”更划算，但摊着本身是免费的——全部是正的的门限项时，“保持摊着”每步收 0，那是一个舒服的盆地。盆地必须是净负的，才逼得出起身。

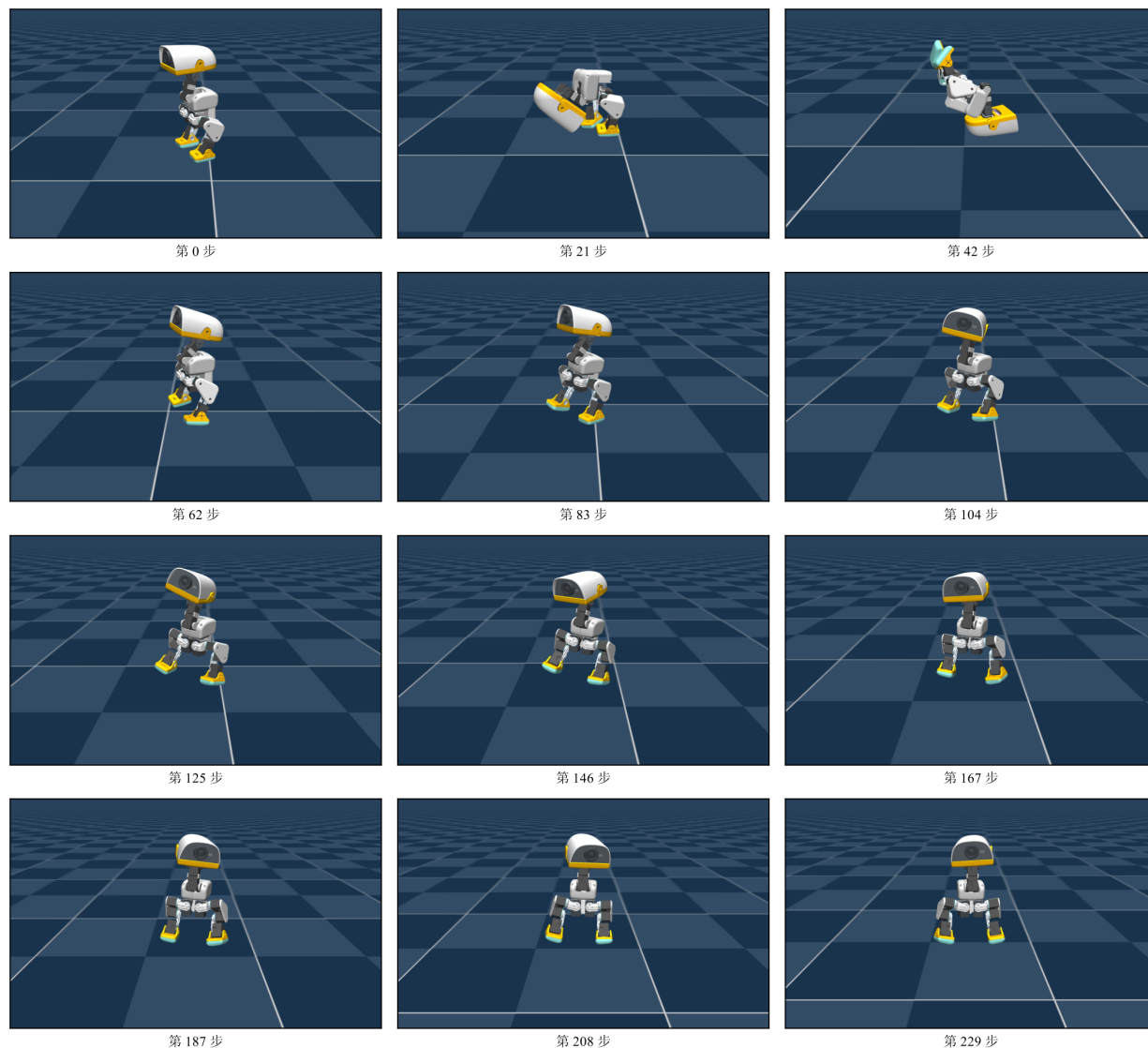


图 24: 前滚翻的一个回合，十二帧覆盖前 230 步（相邻帧 21 步）。第 0 步站立，第 21 步躯干前倾至水平、双脚离地，第 42 步整个倒过来（脚底朝上），第 62 步重新用双脚站住——整个翻滚约 1.2 秒；之后到第 229 步的每一个取样帧上它都站着，没有再翻。

它的对策讲究之处在于带门限，只在翻滚完成之后才生效（按累计转过的角度开门）：翻滚过程本身不上税。惩罚落在“翻完之后该起来却不起来”上，而不落在“正在尝试翻”上——后者会把动作的探索直接掐死。

三、它是十三个平地任务里唯一开对称性增广的。7.10 节会讲那是什么、为什么它必须写在算法侧而不是环境侧。这里先记一句，开它的那个任务，恰好也是唯一严格左右对称的动作。

7.6 目标可以完全不在观测里

前面四个动作有一个共同点容易被忽略：目标从来没有出现在策略的输入里。



图 25: 真机上的踢球：三只小鸭子和一个球。策略的 61 个输入里没有一个是关于球的——球在哪、多大、有没有被踢到，它全看不见。（厂商照片，Pollen Robotics）

踢球那个任务最清楚：面前放一个 70 毫米、15 克的球，奖励的主项是球往前走了多快，而策略的观测还是 1.4 节那 61 个数——球在哪、多大、有没有被踢到，一个都看不到。

它凭什么可解？要它做什么，是奖励说的。球往前走得快就加分，那些恰好让球往前走的动作被反复加强，最后长出来的不是“看见球、去踢它”，而是“在这个固定布局下做一套能把球送出去的动作”。

训练目标由奖励表达；而策略能不能完成任务，还要看观测里有没有完成它所需的信息。这两件事各管一头，缺哪一头都不行：

- **球的位置固定**，策略可以长出一套固定动作——观测里没有球也照样成立，就是眼下这个情形。
- **球的位置会变**，就必须把球的位置、或者别的能用来定位球的信息放进观测。否则策略在观测里根本区分不出“球在左”和“球在右”两种局面，只能对两者输出同一个动作。

这条边界同时也划出了这四个动作的适用范围：观测里没有的东西，策略没法泛化到它。

7.7 “慢才是最优”怎么写进奖励

坐站那个任务要求动作轻、不许砸下去。朴素的做法是给速度加惩罚，但那有个漏洞：惩罚是有限的，而“早点到位、然后一直领到位的分”是按步累加的——冲过去反而更赚。

正确的做法是追一个匀速推进的内部目标：奖励只看“你离那个匀速目标有多近”。冲到目标前面，离匀速目标反而远了，抢跑一分钱好处都没有。慢下来成了最优解，不是因为快被罚，而是因为快没收益。

任何“要求匀速、要求柔和”的动作都能照抄这一条：把“别太快”从一条惩罚改写成一个会跟着时间走的目标。

7.8 写奖励的几条规矩

把这几张奖励表并排放在一起，六条规矩反复出现。每条都配一个上面见过的实例。

(1) 想要的动作写成硬门，不是小额惩罚的暗示。踢球那项“支撑脚必须踩在地上”是硬条件，不满足就不给主项的分。换成“另一只脚离地扣一点分”，策略会算账——扣的比踢球拿的少，于是它用一个双脚离地的凌空动作把球撞出去。欠约束的自由度一定会被利用。

(2) 同一个目标分层写。起身与坐站的姿态和高度都写了两三层：宽的高斯项在离目标很远时提供梯度（不然一开始梯度接近零、学不动），窄的尖峰项负责最后收口，L1 项压掉常值偏差。单用一层要么远处没梯度，要么近处不够准。

(3) 乘性复合打败加性求和。几个条件加起来求和会留下“每项都拿八成”的折中解——躯干歪着、高度差一点、总分不低。改成相乘，任何一项不合格就整体归零。代价是每项的容差要放宽到当前策略够得着分，否则梯度看不见。

(4) 不设一次性大奖。“到达某状态给一大笔”一定会被套利：提前冲到那个状态然后按步领。对策就是 7.7 节那个匀速目标。

(5) 别在坏状态上给正分。“躺着只要头抬起来就给分”——策略会找到最省力的躺姿，把头抬到刚好及格，然后一直躺着领。对策是按进度增量给分：比上一刻更接近目标才给钱。

(6) 正则项分两类，上的时机不一样。挡动作的那类（角速度、角动量、姿态偏离）惩罚的正是动态动作物理上必须做的事，对翻滚、旋转要给得很轻；平滑类（动作变化率、力矩变化率）只挡抖动、可以给得重，但要等动作学会之后再上。走路表里“动作变化太快”从 -0.1 拉到 -1.0 、第 1500 迭代才到位（1.5 节），就是这条的实例——太早拉满，“什么都不做”就成了最优解。

7.9 一个身体，多个策略

1.4 节那个伏笔现在可以收了：所有任务共用同一份 61 维观测布局，用不到的指令槽位填零。

好处是上面每个动作训出的策略，输入输出格式完全一样（61 进 14 出）—— 同一台机器上换掉生效的策略不用改任何接口：走着走着被推倒，切到起身策略；起来再切回行走。代价是每个策略都要接受一些自己用不到的输入。

接口一样是切换的必要条件，不是充分条件。真要在机器上这么切，还得再对齐三件事：各策略的观测归一化与动作尺度一致（否则同一个数字在两个策略里不是一个意思）、每个策略写清**什么状态下才允许切进去**、以及切换瞬间的动作怎么过渡。起身那一档的实际规矩就是：只有当姿态落进它训练时见过的那片分布里才切进去，重新站稳并持续满足稳定判据之后，才平滑切回行走。

7.10 奖励表和课程表都在环境这一侧

奖励表、课程表、域随机化、回合起始状态的混合，全部写在环境配置这一侧，不在算法里。这是 mjlab 这种“管理器式”环境的要害，后果有两个方向：

换算法它们照样生效。第 4-6 节那三份训练脚本换成任何别的 PPO 实现，这些东西一个都不用重写。**换任务算法不用改。**这一整节就是证据：_DEFAULT_TASK 换一行，换一个动作。

一个具体的：走路表里“动作变化太快”的权重由课程表从 -0.1 拉到 -1.0 。这条时间曲线在环境里、不在我们的代码里—— 所以它对 $v1$ 、 $v2$ 、 $v3$ 一视同仁，三档对照才是干净的。

那有没有确实在算法侧的东西？有一个：对称性增广。它把每段经验按左右镜像映射一遍也拿来训（双足机器人是镜像对称的）。要点在于**镜像观测应该产生镜像动作，而不是同一个动作**—— 写成式子是 $\pi(M_o o) = M_a \pi(o)$ ， M_o 与 M_a 分别是观测和动作上的左右互换映射。要求“镜像后输出相同”是错的：那会把左右腿的动作压成同一个值。这两个映射、以及某个任务到底有没有这种对称性，都由环境配置声明。这一项不能写在环境里—— 它改的是 loss，而 loss 属于训练器。所以训练器要自己去问任务要不要它：train_v3_ppo.py 向任务注册表查那个配置，查到就挂上镜像一致性损失，**不在训练器里另存一份”哪个任务要开”的名单**—— 注册表是唯一声明处，抄一份出来，上游哪天改了开关这边会静默失配。

13 个平地主任务逐个查过，**只有前滚翻是开的**，而 7.5 节那一档确实翻出来了。上游为什么给它开：那个动作严格左右对称，镜像一致性正好压住“往一侧塌”这个失败模式（正着翻要经过完全倒立，而往一侧塌是能量更低的斜路）。

一个任务能不能用这一项，判据是它的动力学、奖励、观测、动作与指令分布在镜像变换下是否等变，不是动作看上去左右对不对称。上面那句“唯一开着的恰好是唯一严格对称的动作”说的是这份注册表的配置事实；其余十二个任务把它关着，一样把各自的动作学了出来。**有对称性可利用时这一项才值得付代价**，而它到底为某个动作带来多少，要在同样预算下开关对照才看得出来。

8 本讲小结

8.1 一条演进链：每一步为什么发生

回头看那三级阶梯，每一步都是被上一步的毛病逼出来的：

版本	加了什么	为了解决
v1 REINFORCE	策略梯度 + 常数基线	没有标准答案时怎么调参数
v2 A2C	critic 估值 + GAE 优势	v1 的参照值太粗，把功劳和罪责算错人
v3 PPO	概率比值 clip + 数据复用 + KL 自适应	v2 的更新步子没人管，走坏一步回不来

这条链上没有一步是“因为更先进所以用它”。每一步都对应一个具体的、在同一个任务上看得见的失败。

8.2 一张小结表：同一套算法，几个不同的动作

动作	改了什么	训练预算	拿到了什么
走路（平地）	—— 基准	6000 迭代	评测 0.155、6400 个环境步里零摔倒
轮滑	只换本体（脚 → 被动轮）	2000 迭代	长出完全不同的运动模式：靠上身摆动蹬地，让轮子滚起来
原地旋转	主项换成高转速跟踪	2000 迭代	站在轮上原地转起来
鼻尖取物	目标类型换成空间点	2000 迭代	自己找出“先蹲、再前倾、用膝盖把身体压下去”
前滚翻	主项换成“转过了多少角度”，两项挡动作的正则分别减到走路的 1/25 与 1/20	6000 迭代	站立 → 前倾 → 倒立 → 站回，约 1.2 秒；之后一百多步站着

这张表的“训练预算”一列也是结论的一部分。轮滑、原地旋转、鼻尖取物三个动作只用了 2000 迭代，而它们的课程表末档分别在第 5000、3000、2000 迭代 —— 末档那几级做的都是加宽域随机化，不是搭技能脚手架，所以短跑也出得来动作。前滚翻不一样：它的课程表末档就在 6000，而那几级课程正是把翻滚从“站着不动”里推出来的东西。给一个新动作定预算，看的是它自己那份课程表的末档在第几迭代、以及末档那几级在加噪声还是在搭脚手架。

8.3 还能往哪走

本篇停在“on-policy、无模型、基于策略”这一格。地图上还有两个方向：

- **off-policy**：把经历存进池子反复抽用，样本效率高得多。代价是要处理“用旧数据学新策略”的偏差。
- **后训练**：不从零学，而是拿一个已经会做事的大模型，用强化学习去改进它。

这两条都不构成本篇的前置或后续——读到这里，你已经有一套能自己训出机器人动作的完整流程了。

附录：复现这一讲

A.1 代码、权重与产物

全部代码与交付件在一个公开仓里：

- **仓库**：https://github.com/Xbotics-Embodied-AI-club/RL_duck
- **本篇对应的 tag**：v20260903.01（正文里所有数字都出自这一版）
- **训练器**：`code/train_v1_reinforce.py` · `code/train_v2_a2c.py` · `code/train_v3_ppo.py`
- **环境封装**：`code/env.py` —— 唯一与机器人打交道的地方，换任务只改它顶部一行
- **评测与出图**：`code/rollout.py`（评测）· `code/render_delivery.py`（重出交付图与视频）
- **权重**：交付/权重/ 七份，MANIFEST.json 逐份记了迭代号、观测维度与 sha256
- **视频**：交付/视频/ 十三段（每个动作单只 + 六只，另有一段八档迭代的进化视频）

A.2 环境与硬件

项	取值
Python	3.12.11
PyTorch	2.9.1+cu128 (CUDA 12.8)
仿真器	mjlab 1.3.0 · mujoco 3.12.0 · mujoco-warp 3.8.1 · warp-lang 1.12.0
机器人与任务	mjlab-microduck 0.1.0（机器人模型、奖励表、课程表都来自它）
训练框架	lightning 2.6.5
GPU	NVIDIA A800-SXM4-80GB，单卡一次训练

装法就是仓根一条 `uv sync`：版本全钉在 `uv.lock` 里，不用手动对齐。

A.3 训练与评测口径

项	取值
并行环境数	4096
随机种子	1（三个算法同一个种子，每档一次训练）
一次迭代	采一段 24 步的 rollout，再更新
训练预算	走路三档各 6000 迭代；轮滑 / 旋转 / 取物 2000；前滚翻 6000
控制频率	50 Hz
评测	16 个环境 $\times$ 400 步 = 6400 个环境步，动作取分布均值（不采样）
评测前置	课程表按权重训到的那一档对齐（ <code>rollout.align_curriculum()</code> ）

“评测平均奖励”是那 6400 个环境步上每步总奖励的平均；“摔倒终止”数的是同一段里被终止条件判定为倒下的次数。两个数都由 `code/rollout.py` 打印，不是从训练日志里读的。

正文里的**学习曲线**是另一回事：它读的是训练日志，纵轴是训练时每次迭代的平均奖励。两者不换着用——4.6 节说过为什么。

A.4 参考

- Sutton & Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.
- Williams, “Simple statistical gradient-following algorithms for connectionist reinforcement learning”, *Machine Learning* 8(3–4), 1992. (REINFORCE)
- Mnih et al., “Asynchronous Methods for Deep Reinforcement Learning”, ICML 2016. (A3C / A2C)
- Schulman et al., “High-Dimensional Continuous Control Using Generalized Advantage Estimation”, ICLR 2016. (GAE)
- Schulman et al., “Proximal Policy Optimization Algorithms”, arXiv:1707.06347, 2017. (PPO, 正文那张 clip 图取自它的图 1)
- Achiam, *Spinning Up in Deep RL*, OpenAI, 2018. (第 2 节那张方法分类地图)
- Rudin et al., “Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning”, CoRL 2021. (大规模并行 + 这套腿式奖励表的来路)